

A Major Project Phase II

Report on

**A PYTHON BASED CONTROL SYSTEM FOR
ON DEMAND ENERGY SUPPLY**

Submitted to

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, HYDERABAD

Submitted in partial fulfillment of requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

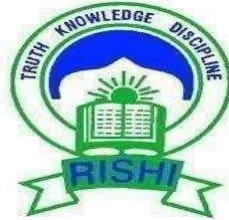
Submitted by

GURANI BHARGAVI 226P1A1222

Under the guidance of

Ms. Sunitha Avula, M. TECH

Assistant professor, Department of IT



DEPARTMENT OF INFORMATION TECHNOLOGY



Rishi M.S Institute of Engineering & Technology for Women

In memory of "BHARAT RATNA" (Late) Smt. M.S.Subbu Lakshmi
(UGC Autonomous)

Approved by AICTE, New Delhi and Affiliated to JNTU-H
Near JNTU Metro Station, Kukatpally, Hyderabad, Telangana – 500 085



2025-2026



Rishi M.S Institute of Engineering & Technology for Women

In memory of "BHARAT RATNA" (Late) Smt. M.S.Subbu Lakshmi

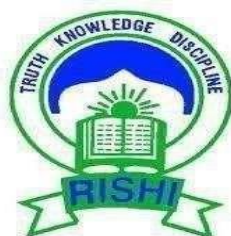
(UGC Autonomous)

Approved by AICTE, New Delhi and Affiliated to JNTU-H

Near JNTU Metro Station, Kukatpally, Hyderabad, Telangana – 500 085



DEPARTMENT OF INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project entitled **"A PYTHON BASED CONTROL SYSTEM FOR ON DEMAND ENERGY SUPPLY"** being submitted by

GURANI BHARGAVI 226P1A1222

In partial fulfilment of the award of degree of **BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY** at **Rishi MS Institute of Engineering & Technology for women**, affiliated to the Jawaharlal Nehru Technological University, Hyderabad during the year 2025-2026. It is a record of Bonafide piece of work, undertaken by the supervision of the undersigned.

INTERNAL GUIDE
Ms. SUNITHA AVULA, M.Tech.
Assistant Professor, Department of IT

HEAD OF THE DEPARTMENT
Ms. R. SRILAKSHMI, M. Tech.
Assistant Professor, Department of IT

Submitted for Viva Voice Examination held on _____

External Examiner _____

ACKNOWLEDGEMENT

This report will certainly not be completed without acknowledgements paid to all those what have helped us in doing our Major Project work.

I would like to express my sincere thanks to my Secretary **Ms. Rajasree** & Academic Director **Ms. Madhavi Latha** for their constant supervision and encouragement, which helped me in completing this work successfully.

I am expressing my sincere gratitude to my Principal **Dr. SK Altaf Hussian Basha**, for his timely suggestions, which helped me to complete this work successfully.

It's my privilege to thank **Ms.R.SriLakshmi**, Head of the Department for her encouragement during the progress of this work.

I derive great pleasure in expressing my sincere gratitude to my Project Coordinator, **Ms. Neela Sruthi** & **Ms.G.Divyabharathi**, for their encouragement and timely suggestions during the progress of this work.

I express my sincere thanks to my Guide **Ms. Sunitha Avula** for giving us moral support, kind attention and valuable guidance to us throughout this work.

I am thankful to my **Rishi MS Institute of Engineering and Technology for Women** for providing the required facilities during the Industrial Project work.

I would like to thank my parents and my friends for being supportive all the time, and I am very much obliged to them.

GURANI BHARGAVI

226P1A1222



Rishi M.S Institute of Engineering & Technology for Women

In memory of "BHARAT RATNA" (Late) Smt. M.S.Subbu Lakshmi

(UGC Autonomous)

Approved by AICTE, New Delhi and Affiliated to JNTU-H

Near JNTU Metro Station, Kukatpally, Hyderabad, Telangana – 500 085



Vision & Mission of the Institution

VISION

- To be a center of excellence in producing women engineers and scientists who are professionally competent social leaders to face multi-disciplinary global environment by imparting quality technical education, values and ethics through innovation methods of teaching and learning.

MISSION

- To promote women technocrats capable enough to resolve the problems faced by the society using the knowledge imparted.
- To prepare self-reliant women engineers for technological growth of the nation and society by laying strong theoretical foundation accompanied by wide practical training.
- To equip the young women with creative thinking capabilities and empowering them towards innovation.



Rishi M.S Institute of Engineering & Technology for Women

In memory of "BHARAT RATNA" (Late) Smt. M.S.Subbu Lakshmi

(UGC Autonomous)

Approved by AICTE, New Delhi and Affiliated to JNTU-H

Near JNTU Metro Station, Kukatpally, Hyderabad, Telangana – 500 085



B. Tech IT – Major Project

Course Outcomes (COs)

CO	Course Outcome Description
CO1	Identify and analyze a real-world problem and define project objectives and scope.
CO2	Apply engineering knowledge and modern tools to design an appropriate solution.
CO3	Implement the designed solution using suitable programming languages and technologies.
CO4	Test, evaluate, and validate the developed system for performance and reliability.
CO5	Demonstrate teamwork, project management, documentation, and effective communication skills.

CO – PO – PSO Mapping Table

(1-Low, 2-Medium, 3- High)

COs \ POs	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	2	3	2	–	2	–	–	–	2	–	2	2	2	–
CO2	3	2	3	2	3	–	–	–	2	–	2	2	3	2
CO3	3	2	3	–	3	–	–	–	2	–	2	2	3	3
CO4	2	2	2	3	2	–	–	–	2	–	2	2	2	2
CO5	2	2	2	–	2	–	–	2	3	2	3	2	–	–



Rishi M.S Institute of Engineering & Technology for Women

In memory of "BHARAT RATNA" (Late) Smt. M.S.Subbu Lakshmi

(UGC Autonomous)

Approved by AICTE, New Delhi and Affiliated to JNTU-H

Near JNTU Metro Station, Kukatpally, Hyderabad, Telangana – 500 085



DEPARTMENT OF INFORMATION TECHNOLOGY

Vision & Mission of the Department

Vision of the Department

To promote center of excellence in sustainable academic environment to impart technical skills and strong research platform to prepare global leaders

Mission of the Department

1. To create good academic platform with modern teaching and learning methodologies.
2. To enhance leadership qualities with ethics, values and sense of teamwork for professional excellence.
3. To inculcate research culture by encouraging projects in advanced technologies through industry interactions.

PROGRAM OUTCOMES(POs)

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review search literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



Rishi M.S Institute of Engineering & Technology for Women

In memory of "BHARAT RATNA" (Late) Smt. M.S.Subbu Lakshmi

(UGC Autonomous)

Approved by AICTE, New Delhi and Affiliated to JNTU-H

Near JNTU Metro Station, Kukatpally, Hyderabad, Telangana – 500 085



Department of Information Technology

Program specific outcomes (PSO's)&Program Educational objectives (PEO'S)

Program specific outcomes (PSO's)

PSO 1: Improve the student's ability to decipher the basic principles and methodology of computer systems. Improve the students' ability to absorb facts and technical ideas in order to build and develop software.

PSO 2: The capacity to create novel job routes as an entrepreneur using modern computer languages and evolving technologies like SDLC, Python, Machine Learning, Social Networks, Cyber Security, Mobile Apps etc.

Program Educational objectives (PEO'S)

PEO – 1: Engineering graduates with excellent fundamental and technical skills will have successful careers in industry, meeting the needs of Indian and worldwide firms.

PEO – 2: With determination, development, self-reliance, leadership, morality, and moral principles, engineering graduates will become successful entrepreneurs who will leverage employability.

PEO – 3: To support personal and organizational progress, engineering graduates will pursue higher education and engage in lifelong learning.

**A PYTHON BASED
CONTROL SYSTEM FOR
ON DEMAND ENERGY SUPPLY**

TABLE OF CONTENTS

S.NO	CONTENTS	PGNO
i	Abstract	i
ii	List Of figures	ii
iii	List of Acronyms	iii
1	INTRODUCTION 1.1 Problem Statement 1.2 Scope of research 1.3 Existing System 1.3.1 Disadvantages of existing system 1.4 Proposed System 1.4.1 Advantages of proposed system 1.5 Organization of the report	1 - 5
2	LITERATURE REVIEW	6- 7
3	METHODOLOGIES 3.1 System Architecture	8– 12
4	SOFTWARE REQUIREMENTS SPECIFICATIONS 4.1 Functional Requirements 4.2 Performance Requirements 4.3 Software Requirements 4.4 Hardware Requirements	13-15
	4.5 Languages/Technologies Used	16-19

5	SYSTEM DESIGN 5.1 Introduction To UML 5.2 UML diagrams 5.2.1 Data Flow Diagram 5.2.2 Use Case Diagram 5.2.3 Class Diagram 5.2.4 Activity Diagram 5.2.5 Sequence Diagram	20-26
6	IMPLEMENTATION AND RESULTS 6.1 Code	27-38
7	TESTING 7.1 System Testing 7.2 Types of testing 7.2.1 Unit Testing 7.2.2 Integration Testing 7.2.3 Black Box Testing 7.2.4 White Box Testing 7.2.5 Performance Testing 7.2.6 Acceptance Testing 7.3 Test Cases	39-42
8	RESULTS	43-52
9	CONCLUSION	53
10	FUTURE ENHANCEMENT	54
11	REFERENCES	55

ABSTRACT

A PYTHON BASED CONTROL SYSTEM FOR ON DEMAND ENERGY SUPPLY is a modern web-based solution designed to digitally transform the traditional fuel distribution process by enabling users to request fuel delivery directly to their location through an online platform. In many real-world situations, such as vehicle breakdowns, long queues at fuel stations, emergency travel needs, or users located in remote areas, accessing fuel becomes challenging and time-consuming. Existing fuel distribution methods depend heavily on manual processes and physical presence at fuel stations, resulting in inefficiencies, delays, and limited transparency.

Fuel stations are provided with dedicated dashboards to manage fuel inventory, approve requests, assign delivery employees, and monitor transaction records. Delivery employees receive assigned tasks digitally and update delivery status upon completion, ensuring smooth operational flow and accountability. The administrative module plays a supervisory role by monitoring users, fuel stations, transactions, and feedback while maintaining overall system security and efficiency.

The system is developed using modern web technologies, including **HTML, CSS, and JavaScript** for the frontend interface, while the backend logic is implemented using **Python with the Flask framework**. **MySQL** is used as the relational database for secure data storage and management. Role-based access control, secure authentication mechanisms, and structured workflow management ensure reliability and data integrity throughout the application. The system emphasizes automation, transparency, and real-time communication between stakeholders, reducing manual errors and enhancing operational productivity.

Overall, this project **A Python Based Control System for On Demand Energy Supply** provide an innovative approach to fuel distribution by combining digital automation with practical delivery management. It improves customer convenience through doorstep fuel delivery, enables efficient resource management for fuel stations, and introduces scalable architecture suitable for future enhancements such as GPS tracking, real payment gateway integration, and intelligent analytics. The project demonstrates how web-based technologies can be effectively utilized to solve real-world logistical challenges and improve service accessibility in a technologically evolving environment.

LIST OF FIGURES

Fig. No	Figure Name	Page. No
3.1	System Architecture	9
5.2.1	Data Flow Diagram	21
5.2.2	Use Case Diagram	22
5.2.3	Class Diagram	24
5.2.4	Activity Diagram	25
5.2.5	Sequence Diagram	26
8.1.10	Login Page	43
8.1.11	Registration Page	44
8.1.12	Admin Page	46
8.1.15	Fuel page	48
8.1.19	Results	52

LIST OF ACRONYMS

SNO	Acronyms	Description
1	UML	Unified Modified Language
2	API	Application Programming Interface
3	DB	Data Base
4	HTML	Hyper Text Markup Language
5	CSS	Cascading Styling Sheet
6	URL	Uniform Resource Locator
7	JS	JavaScript

CHAPTER - 01

INTRODUCTION

INTRODUCTION

Fuel is one of the most essential resources for transportation, industrial operations, agriculture, logistics, and emergency services. In everyday life, vehicles and machines depend heavily on a continuous fuel supply to function efficiently. Traditionally, users must travel to fuel stations to refill fuel, which can often be inconvenient, especially during emergencies such as vehicle breakdowns, traffic congestion, long queues, or situations where fuel stations are located far from the user's location. The concept of online delivery services has already proven successful in sectors such as food delivery, e-commerce, and logistics. Applying the same concept to fuel distribution opens opportunities to simplify fuel access and improve service quality. **A Python based control system for on demand energy supply** is designed to provide a web-based platform where users can request fuel delivery directly to their location through an online interface. The system connects users, fuel stations, delivery employees, and administrators within a single structured environment. It ensures that fuel requests are managed efficiently through digital workflows including request approval, payment processing, employee assignment, and delivery tracking. This system focuses on reducing the dependency on manual processes and improving transparency in fuel distribution. Users can track their orders in real time, fuel stations can manage inventory and requests systematically, and administrators can monitor the overall system operations. By integrating modern web technologies and database management, the system aims to provide a reliable, scalable, and user-friendly solution that addresses real-world fuel accessibility challenges.

1.1 PROBLEM STATEMENT

The traditional fuel distribution model presents several challenges that affect both customers and service providers. Customers face inconvenience due to the need to travel to fuel stations, especially during emergencies or remote travel situations. There is no dedicated online system to request fuel delivery or monitor delivery status. Manual record keeping causes inefficiencies, data errors, and lack of proper monitoring. Fuel stations struggle with managing requests efficiently,

and employee assignments are often unstructured. Additionally, the absence of real-time communication and centralized management leads to poor transparency and reduced customer satisfaction. Hence, there is a need for a secure, efficient, and scalable online system that automates fuel ordering, delivery management, and operational monitoring.

1.2 SCOPE OF RESEARCH

The scope of the Online Fuel Delivery and Management System includes the design and implementation of a web-based platform that manages the complete fuel delivery lifecycle. The system supports multiple user roles such as administrators, fuel stations, employees, and customers, each with dedicated functionalities. The system covers user registration, fuel request placement, request approval, online payment processing, delivery task assignment, status tracking, and feedback management. It ensures centralized data storage and secure authentication while maintaining role-based access control. The project is limited to web-based implementation and focuses on efficient fuel delivery management within the defined operational environment. Future extensions may include mobile applications, real-time GPS tracking, and integration with advanced analytics and payment systems.

1.3 EXISTING SYSTEM

In the existing fuel distribution system, customers are required to visit fuel stations physically to purchase fuel. The entire process is mostly manual, where fuel sales, inventory tracking, and employee management are handled without centralized automation. Delivery services are rarely available, and there is no structured mechanism for emergency fuel supply. Record maintenance in many cases is performed manually, which increases the possibility of errors and data inconsistency. Customers have no option to track fuel availability or delivery status, and communication between customers and fuel providers is limited. Employee task allocation is often unorganized, leading to inefficient service management. Overall, the current system lacks digital integration, automation, and real-time monitoring capabilities.

1.3.1 Disadvantages of Existing System

➤ Cannot Prevent Internal Threats

Firewalls are mainly designed to protect against external threats coming from outside the organization. However, they are less effective against internal threats such as employees.

➤ **2. Limited Protection Against Advanced Attacks**

Basic firewall systems may not be capable of detecting sophisticated cyber-attacks such as advanced malware, phishing, ransomware, or application-level attacks.

➤ **Performance Issues**

Firewalls inspect every data packet that passes through the network, which can consume system resources and processing power.

1.4 PROPOSED SYSTEM

The proposed Online Fuel Delivery and Management System introduce a fully automated and web-based solution to overcome the limitations of the existing system. The system enables users to register and place fuel delivery requests online without needing to visit fuel stations physically. Once a request is placed, fuel stations can review and approve requests, manage fuel availability, and assign delivery employees through the system. Users can make online payments and track delivery progress in real time. Delivery employees receive task information digitally and update delivery status after completing the service. The administrator manages the overall platform, including user management, fuel station approvals, and system monitoring. The proposed system ensures better coordination between stakeholders, reduces manual workload, improves transparency, and enhances customer convenience through doorstep fuel delivery.

1.4.1 ADVANTAGES OF PROPOSED SYSTEM

➤ **Enhanced Security Protection**

The system provides stronger security compared to traditional firewalls by using advanced techniques such as deep packet inspection and intrusion prevention. It can detect and block complex cyber threats more effectively.

➤ **Real-Time Threat Detection**

The system continuously monitors network traffic and identifies suspicious activities in real time. This helps in quickly detecting and preventing potential cyber-attacks before they cause damage.

➤ **Improved Traffic Control**

The firewall system offers better control over network traffic by using dynamic and flexible security rules. It allows only authorized traffic while blocking harmful data.

➤ **Better Performance Optimization**

The system is designed to handle large volumes of network traffic efficiently without significantly affecting performance. It ensures smooth and fast network operations.

1.4.2 ORGANIZATION OF THE REPORT

This project report is systematically organized to provide a clear and detailed understanding of the development of a Python-Based Control System for On-Demand Energy Supply. The report is structured to give readers a comprehensive understanding of the system, from the motivation behind the project to its implementation and future scope. The report is organized as follows:

1. Introduction

This section introduces the concept of energy management and highlights the challenges in traditional power distribution systems, such as energy wastage, inefficient usage, and lack of real-time control. It explains the motivation behind developing a Python-based control system for efficient energy usage.

2. Literature Review

This chapter discusses existing research, systems, and technologies related to energy management and smart energy systems. It highlights the limitations of current systems, such as lack of automation, poor demand prediction, and inefficient energy distribution, and explains how the proposed system addresses these challenges.

3. System Analysis

This section explains the system requirements and problem definition along with an analysis of existing systems. It examines the drawbacks of traditional energy supply methods and identifies the need for an intelligent, automated, and demand-based energy control system.

4. Proposed System

The proposed system's architecture, features, and workflow are described in detail. This chapter explains how Python is used to control and manage energy supply based on demand, including real-time monitoring and decision-making.

5. Methodology

This section describes the working process of the system, including data collection from sensors, processing using Python algorithms, and controlling energy distribution. It also explains how the system analyzes demand and adjusts supply accordingly.

6. System Design and Implementation

The design phase includes system architecture diagrams, data flow diagrams, and interface design. Implementation discusses the technologies used such as Python, IoT devices, sensors, and control modules, and explains how all components are integrated.

7. Results and Discussion

This section presents the performance of the system, efficiency in energy usage, and accuracy in demand-based supply. It includes analysis using relevant metrics and comparison with traditional systems.

8. Conclusion and Future Work

Summarizes the key findings, benefits, and contributions of the project. It also suggests future improvements such as integration with renewable energy sources, AI-based prediction models, and smart grid technologies.

9. References

Lists all the academic papers, websites, datasets, tools, and books referred to during the development and research of the project.

CHAPTER - 02

LITERATURE REVIEW

Title: IoT-Based Smart Energy Management System

Author: A. Sai Kiran et al.,2024

This system utilizes Internet of Things (IoT) technology to enable real-time monitoring and control of energy consumption. Smart sensors and meters are installed in devices to continuously collect data such as voltage, current, and usage patterns. This data is transmitted to a centralized system where it is analysed to identify energy demand trends. Based on this analysis, the system automatically adjusts energy distribution, turning devices on or off as needed. It helps reduce unnecessary energy consumption, lowers electricity costs, and improves overall energy efficiency. Additionally, users can monitor their energy usage remotely through web or mobile applications, making the system more user-friendly and effective.

Title: Demand Response System Using Machine Learning

Author: Srushti Gunthe et al., 2023

This system incorporates machine learning techniques to predict future energy consumption based on historical data. Algorithms such as regression models or decision trees analyse past usage patterns, peak hours, and user behaviour. Based on these predictions, the system dynamically adjusts energy supply to match demand, ensuring optimal utilization of resources. For example, during peak hours, the system can reduce power consumption of non-essential devices or shift loads to off-peak times. This approach not only enhances energy efficiency but also reduces strain on power grids and prevents energy shortages. Over time, the system improves its accuracy by learning from new data, making it more intelligent and adaptive.

Title: Smart Automation Petrol Pump System Based on IOT

Author: R. Gowri & R. Sathish Kumar et al. 2022

This system is based on smart grid technology, which modernizes traditional power distribution systems by integrating digital communication and automation. It collects real-time data from various nodes in the grid, such as substations, transformers, and consumer endpoints. Using this data, the system can automatically detect faults, manage load distribution, and ensure efficient energy flow. It prevents issues like overloading and power outages by balancing the supply across

different regions. Smart grids also support integration with renewable energy sources, allowing efficient use of solar and wind power. Overall, the system improves reliability, stability, and efficiency of energy distribution.

Title: Automated Energy Control System Using Python

Author: R. Khatun et al.2021

This system focuses on automating energy control using Python programming. It integrates hardware components like sensors, microcontrollers, and relays with software algorithms written in Python. The system collects real-time data from connected devices and processes it to make decisions about energy usage. For instance, it can automatically switch off appliances when not in use or adjust power levels based on requirements. Python libraries and frameworks are used to handle data processing, communication, and control operations. This reduces the need for manual monitoring and improves efficiency. The system is flexible, scalable, and can be easily integrated with web applications for remote control and monitoring.

5. Title: Renewable Energy Monitoring and Control System

Author: Singh, A. et al,2020

This system is designed to monitor and manage renewable energy sources such as solar panels and wind turbines. It continuously tracks parameters like energy generation, weather conditions, and storage levels. Based on this data, the system optimizes the usage of renewable energy by balancing supply with demand. For example, excess energy generated during the day can be stored in batteries and used later when production is low. The system ensures maximum utilization of clean energy while minimizing dependence on conventional power sources. It also helps in reducing environmental impact and promotes sustainable energy practices.

Title: Unmanned Fuel System Using RFID Tools

Author: S. Rohini et al.,2020

This system focuses on optimizing energy consumption using Artificial Intelligence techniques. It collects real-time data from smart devices and RFID tools analyses usage patterns using AI algorithms such as neural networks and predictive models. Based on this analysis, the system automatically adjusts energy distribution by prioritizing essential loads.

CHAPTER - 03

METHODOLOGY

3.1 SYSTEM ARCHITECTURE

The system architecture for the Python-based control system for on-demand energy supply is designed to efficiently monitor, analyze, and control energy usage in real time. It consists of multiple interconnected components that work together to ensure optimal energy distribution based on user demand. In addition to basic monitoring and control, the architecture focuses on building an intelligent and automated energy management system that minimizes energy wastage and maximizes efficiency. The system is capable of continuously observing energy consumption patterns and making real-time decisions without requiring constant human intervention. By integrating modern technologies such as IoT, data analytics, and Python-based automation, the system becomes highly responsive and adaptive to changing energy needs. The architecture supports both centralized and distributed control mechanisms. In a centralized approach, all data is collected and processed in a single system, whereas in a distributed approach, some decision-making can happen at the device level itself. This flexibility makes the system scalable and suitable for different environments such as homes, industries, and smart buildings. Another important aspect of the architecture is its ability to handle peak load conditions. During high energy demand periods, the system intelligently prioritizes essential devices while reducing or temporarily shutting down non-essential loads. This demand-based control helps in preventing overloads, reducing electricity bills, and improving overall system stability. The system also incorporates predictive capabilities, where historical data is analysed to forecast future energy consumption. This allows the system to prepare in advance and allocate resources more efficiently. For example, energy usage trends can be used to optimize power distribution during specific times of the day.

BENEFITS OF USING SYSTEM ARCHITECTURE

1. Efficient Energy Utilization

The system architecture helps in optimizing energy usage by monitoring and controlling power consumption in real time. It ensures that energy is supplied only when required, reducing wastage.

2. Real-Time Monitoring and Control

It allows continuous tracking of energy consumption through sensors and smart devices. Users and the system can make instant decisions to control devices based on demand.

3. Automation of Energy Management

The architecture supports automation using Python algorithms. Devices can be automatically turned ON/OFF without human intervention, improving convenience and efficiency.

4. Scalability

The system can be easily expanded by adding more sensors, devices, or modules. It is suitable for small homes as well as large industries.

5. Cost Reduction

By reducing unnecessary energy consumption and optimizing usage, the system helps in lowering electricity bills and operational costs.

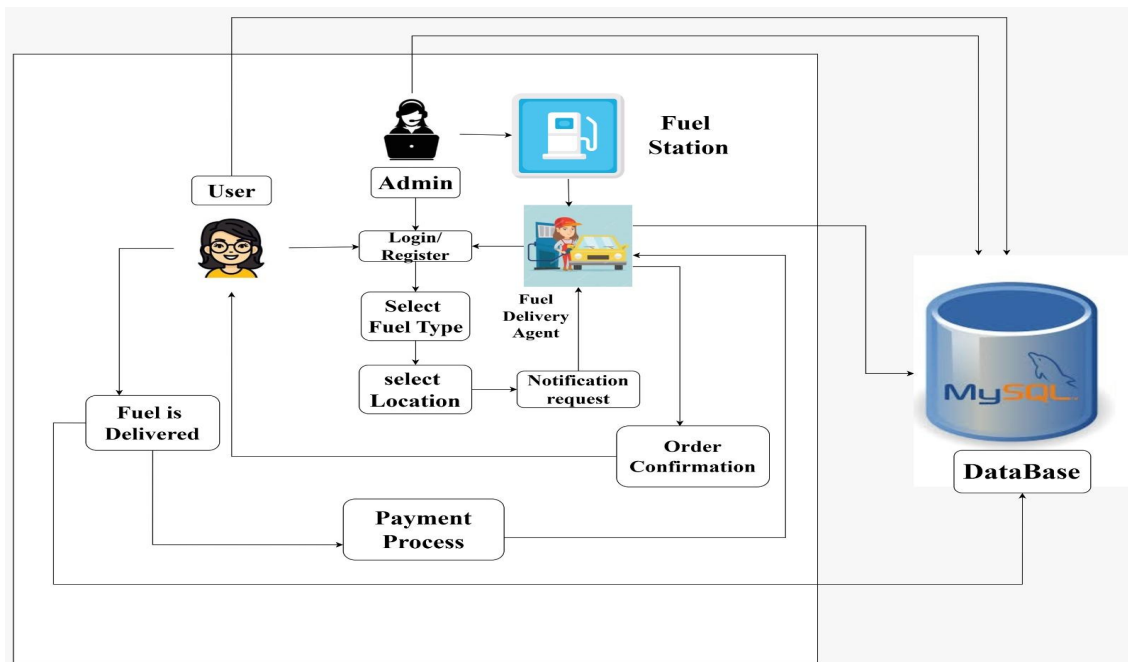


Fig 3.1 System Architecture

SYSTEM ARCHITECTURE EXPLANATION

1. User Interaction

- The system begins with the user accessing the application through a web or mobile interface.
- The user completes registration/login to access the system securely.
- After logging in, the user selects the fuel type (petrol/diesel) and enters the delivery location.
- The system provides available options and estimated delivery details.
- Finally, the user proceeds to the payment process to place the order.

2. Admin Role

- The admin acts as the central controller of the system.
- Responsible for managing user data, orders, payments, and delivery operations.
- The admin monitors system performance and ensures smooth functioning.
- Can update fuel prices, manage fuel station details, and handle issues or complaints.

3. Fuel Station

- Once the order is placed, the request is forwarded to the fuel station.
- The fuel station verifies fuel availability and prepares the order.
- It coordinates with delivery agents to ensure timely delivery.
- Ensures quality and quantity of fuel before dispatch.

4. Delivery Agent

- The delivery agent receives a notification request from the system.
- After accepting the request, the agent collects fuel from the station.
- The agent navigates to the user's location using GPS support.
- Ensures safe and secure transportation of fuel.
- Updates delivery status in real time.

5. Order Processing

- The system generates an order confirmation after successful payment.
- All order details such as fuel type, quantity, location, and time are processed.
- The system tracks the order at each stage (processing, dispatched, delivered).
- Notifications are sent to users regarding order status.

6. Payment Process

- Users can make payments through digital methods like UPI, debit/credit cards, or wallets.
- Payment status is verified before confirming the order.

7. Database (MySQL)

- The database stores all important data including:
 - User information
 - Order details
 - Payment records
 - Delivery status
- It helps in maintaining data consistency and supports future analysis.
- Ensures quick retrieval of information when required.

8. Notification System

- The system sends notifications to both users and delivery agents.
- Users receive updates about order confirmation, dispatch, and delivery.
- Delivery agents receive alerts for new orders and instructions.

9. Final Delivery

- The delivery agent successfully delivers fuel to the user's location.
- The system updates the order status as completed.
- Users may provide feedback or ratings for service improvement.

ADVANTAGES

1. Energy Management - Optimizes energy usage based on real-time demand and reduces unnecessary power consumption.

2. Real-Time Monitoring - Tracks energy usage continuously using sensors and also it Provides instant updates to users and system.

3. Automation - Automatically controls devices (ON/OFF) without manual effort. Saves time and reduces human intervention.

4. Cost Saving- Minimizes electricity bills by avoiding energy wastage and improves overall operational efficiency.

5. Scalability - Can be easily expanded to include more devices and users. Suitable for homes, industries, and smart cities.

DISADVANTAGES

1. High Initial Cost - Requires investment in sensors, hardware, and setup.

2. Complex Implementation - Integration of hardware, software, and networking can be difficult.

3. Dependency on Internet - System performance depends on stable internet connectivity.

4. Security Risks - IoT systems may be vulnerable to cyber-attacks if not properly secured.

5. Maintenance Requirement - Requires regular maintenance and updates for proper functioning.

CHAPTER - 04

SOFTWARE REQUIREMENTS SPECIFICATIONS

4.1 FUNCTIONAL REQUIREMENTS

Functional requirements describe the core functionalities and operations that the system must perform to meet user needs and achieve project objectives.

Admin Functions

- Admin login authentication
- Approve or reject fuel station registrations
- Manage fuel categories and pricing details
- View and manage registered users
- Monitor all transactions and system activities
- View feedback and provide replies
- Manage overall system operations

User Functions

- User registration and login
- Profile management
- Search available fuel stations
- Place fuel delivery requests
- Make online payments
- Track delivery status
- Confirm delivery completion
- Submit feedback and ratings

Fuel Station Functions

- Fuel station registration and admin approval
- Add and manage fuel types
- Update fuel availability and price
- View incoming fuel requests

- Approve user requests
- Assign delivery employees
- Monitor order and payment status

Employee Functions

- Employee login authentication
- View assigned delivery tasks
- Update delivery progress
- Mark delivery as completed
- View completed task history
- Manage account settings

Payment Functions

- Secure payment validation
- Update payment status
- Maintain transaction records
- Support online payment workflow
- The system must operate continuously without failure.
- It should handle errors and recover automatically.

4.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints of the system that ensure reliability, performance, and usability.

Performance Requirements

- Fast response time for user requests
- Efficient handling of multiple users simultaneously
- Smooth data retrieval and updates

Security Requirements

- Secure login authentication
- Role-based access control
- Protected user data and transactions

Usability Requirements

- Simple and user-friendly interface
- Easy navigation across modules
- Clear workflow for placing fuel orders

Reliability Requirements

- Consistent system performance
- Accurate data storage and retrieval
- Reduced system failures and errors

Scalability Requirements

- Ability to handle increased users and fuel stations
- Flexible architecture for future upgrades

Maintainability Requirements

- Structured code organization
- Easy debugging and updates
- Modular system design

4.3 HARDWARE REQUIREMENTS

Hardware requirements define the physical components necessary for the implementation and operation of the Python-based control system for on-demand energy supply. These components include computing devices, sensors, and communication modules that work together to collect, process, and control energy data. Proper hardware selection ensures efficient system performance, accurate data collection, and reliable operation. The hardware setup plays a crucial role in enabling real-time monitoring and automation of energy usage.

- Processor (CPU) - intel i7
- RAM - 8GB
- Storage - 256GB
- Internet Connection - Broadband connection

4.4 SOFTWARE REQUIREMENTS

Software requirements specify the programs, tools, and technologies needed to develop and run the system. In this project, software components are responsible for data processing, decision-making, user interaction, and system control. The use of Python and related frameworks enables

efficient implementation of automation and real-time analysis. These requirements ensure that the system operates smoothly, provides a user-friendly interface, and supports integration with hardware components and database

- Frontend - HTML, CSS, Bootstrap and JavaScript
- Programming Languages – Python (Flask)
- Data Base – My SQL
- IDE – VS Code
- Operating System - Windows 11

4.5 LANGUAGES/TECHNOLOGIES

The development of the Python-based control system for on-demand energy supply is supported by a combination of modern technologies and programming languages that ensure efficiency, reliability, and scalability. These technologies are carefully selected to handle different aspects of the system such as user interaction, data processing, storage, and system control.

The frontend of the system is designed using HTML, CSS, Bootstrap, and JavaScript, which together provide a responsive, interactive, and user-friendly interface. HTML is used to structure the web pages, CSS enhances the visual appearance, Bootstrap ensures responsiveness across different devices, and JavaScript enables dynamic behaviour and real-time updates on the user interface. This combination allows users to easily monitor and control energy usage through an intuitive platform.

The backend is developed using Python with the Flask framework, which plays a crucial role in handling the core logic of the system. Flask is a lightweight and flexible web framework that facilitates communication between the frontend and backend. It processes user requests, interacts with the database, and executes control algorithms for energy management. Python is chosen due to its simplicity, powerful libraries, and strong support for automation, making it ideal for implementing intelligent control systems.

For data storage and management, MySQL is used as the database. It efficiently stores important information such as user details, device status, energy consumption data, and system logs. The database ensures data consistency, security, and quick retrieval, which is essential for real-time monitoring and analysis.

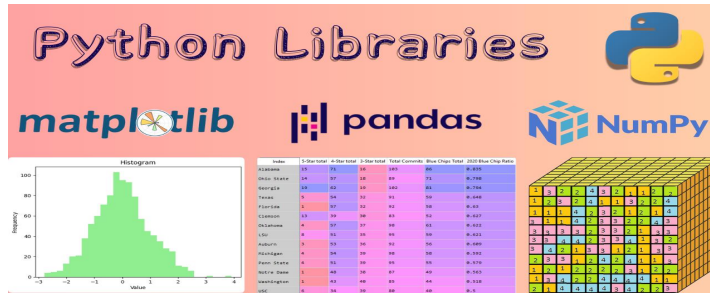


Fig 4.5 Libraries

The project is developed using Visual Studio Code (VS Code), a widely used Integrated Development Environment (IDE) that provides features such as code editing, debugging, extensions, and version control support. It improves developer productivity and simplifies the development process.

The system operates on the Windows 11 operating system, which provides a stable and user-friendly environment for development and execution. It supports all required tools, libraries, and frameworks needed for the project.

4.6 TECHNOLOGIES

Frontend Technologies

The frontend of the system is developed using HTML, CSS, Bootstrap, and JavaScript to provide an interactive and user-friendly interface. HTML (Hypertext Markup Language) is used to design the basic structure of web pages, allowing the arrangement of elements such as forms, buttons, and data displays. CSS (Cascading Style Sheets) is used to enhance the visual appearance of the application by adding colors, layouts, and styling features, making the interface more attractive and easier to use.

1.HTML

HTML is used to create the basic structure of the web pages, including login forms, dashboards, and data display sections. In this project, HTML is used as the foundation for developing the web application. It defines the overall structure and layout of the system by organizing elements such as headings, forms, input fields, buttons, and data sections. HTML is used to create important pages like user registration, login, and the energy monitoring dashboard. It enables proper arrangement of content so that users can easily interact with the system. Additionally, HTML plays a key role in displaying real-time energy data in a structured format, making it understandable for users.

2.CSS

In this project, CSS is used to enhance the visual design and presentation of the web application. It is responsible for styling elements created using HTML by adding colours, fonts, spacing, borders, and layout designs. CSS improves the readability and attractiveness of the interface, ensuring a better user experience. It also helps in organizing the content in a clean and structured way, making the system easy to navigate. Advanced CSS features are used to create responsive layouts and maintain consistency across all pages of the application.

3.JAVASCRIPT

In this project, JavaScript is used to add interactivity and dynamic functionality to the web application. It allows the system to respond to user actions such as button clicks, form submissions, and input validation. JavaScript is also used to update energy consumption data in real time without refreshing the page. It helps in displaying alerts, notifications, and dynamic content, improving user engagement. By handling client-side operations efficiently, JavaScript enhances the overall performance and responsiveness of the system.

4.BOOTSTRAP

Bootstrap is used to simplify the design process and ensure responsiveness. It provides pre-designed components such as navigation bars, buttons, forms, and grid systems, which help in building the interface quickly and efficiently. Bootstrap ensures that the application automatically adjusts to different screen sizes, making it accessible on desktops, tablets, and mobile devices. It reduces development time and ensures a professional look for the system without requiring complex design efforts.

BACKEND TECHNOLOGIES

- **Python**

In this project, Python is used as the core programming language to implement system logic and data processing. It handles tasks such as collecting data from sensors, analysing energy usage, and making decisions for controlling devices. Python is also used to automate processes like switching devices ON/OFF based on demand. Its simplicity and powerful libraries make it suitable for developing intelligent and efficient control systems.

- **Flask Framework**

In this project, Flask is used as a backend framework to develop the web application and manage communication between different components. It handles user requests, processes input data. It is mainly used to create routes (URLs) that connect the frontend with backend logic. Flask also helps in handling form submissions, managing sessions, and integrating with databases for storing and retrieving data. With its built-in development server and debugging support, it simplifies testing and development.

CHAPTER - 05

SYSTEM DESIGN

Systems design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. One could see it as the application of systems theory to product development. Object-oriented analysis and design methods are becoming the most widely used methods for computer systems design.

5.1 INTRODUCTION TO UML

A UML diagram is a diagram based on the UML (Unified Modelling Language) with the purpose of visually representing a system along with its main actors, roles, actions, artifacts or classes, in order to better understand, alter, maintain, or document information about the system. The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML. The Unified Modelling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modelling and other non- software systems. The UML represents a collection of best engineering practices that have proven successful in the modelling of large and complex systems. The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects and

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modelling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modelling language.
5. Encourage the growth of OO tools market.
6. Integrate best practices.

5.2 UML DIAGRAMS

5.2.1 DATAFLOW DIAGRAM

The Data Flow Diagram (DFD) of the python-based control system for on demand energy supply illustrates how data moves through the application from the user input stage to the final result. The process begins when the user logs into the system by providing their login or registration details. These credentials are verified with the user database before the user is allowed to proceed. A Data Flow Diagram (DFD) is a graphical representation that shows how data flows through a system. It describes how data is input, processed, stored, and output within the system. DFD helps in understanding the overall working of the system by breaking it into smaller processes and showing interactions between users, processes, and databases. In this project, the DFD explains how user requests, fuel orders, and payment information flow through the system and how different components interact with each other. This request is processed and sent to the delivery agent as a notification. After the agent accepts the request, the system generates order confirmation and proceeds with the payment process. The payment information is verified, and the transaction status is updated. Finally, the delivery agent delivers the fuel, and the system updates the order status. Throughout this process, all data such as user details, orders, and payments are stored in the database and accessed whenever required.

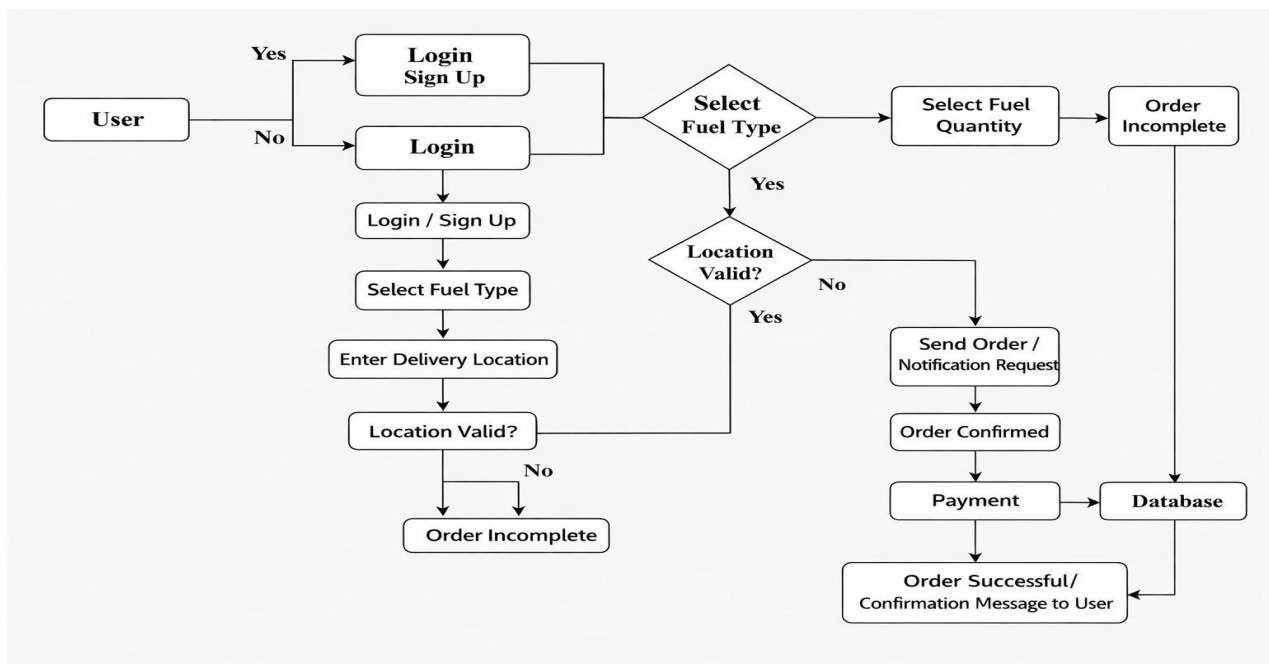


Fig 5.2.1 Data Flow Diagram

5.2.2 USE-CASE DIAGRAM

A Use Case Diagram is a visual representation of how different users (actors) interact with a system. It helps in understanding the functional requirements of the system by showing the relationships between users and various system operations. In this project, the use case diagram illustrates how the user, admin, and fuel delivery agent interact with the online fuel delivery system.

The diagram provides a clear overview of system functionalities such as registration, fuel selection, order placement, payment processing, and delivery. It simplifies the understanding of system behaviour and helps in designing and developing the application effectively.

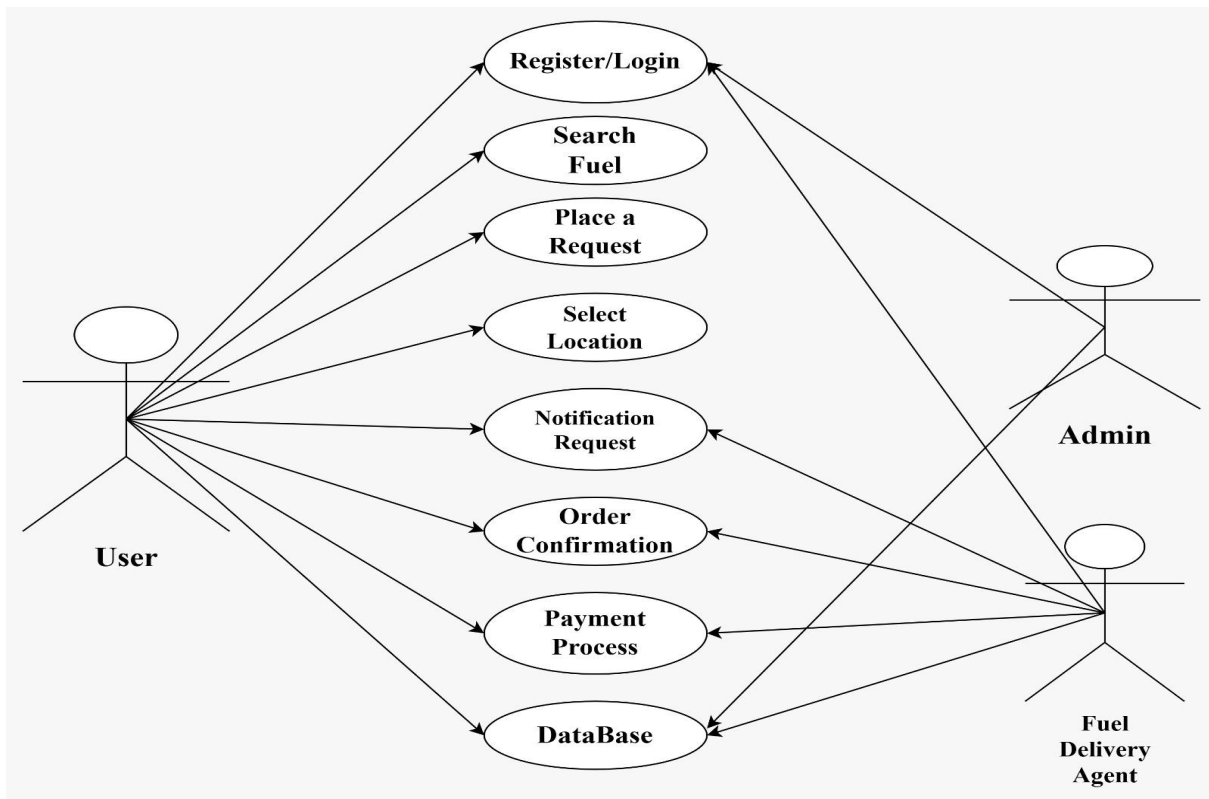


Fig 5.2.2 Use Case Diagram

Step 1: User Registration/Login

- The process starts with the user and the user must register or log in to access the system.

Step 2: Search Fuel

- After login, the user searches for available fuel types (petrol/diesel).
- The system displays available options.

Step 3: Place a Request

- The user places a request for fuel delivery.
- The request includes details like quantity and preferences.

Step 4: Select Location

- The user selects the delivery location.
- This helps the system identify where fuel should be delivered.

Step 5: Notification Request

- The system sends a notification to the fuel delivery agent.
- The agent is informed about the new order.

Step 6: Order Confirmation

- Once the request is accepted, the system confirms the order.
- The user receives confirmation details.

Step 7: Payment Process

- The user proceeds to payment.
- Payment is completed using online methods like UPI or cards.

Step 8: Database Interaction

- All details (user info, orders, payments) are stored in the database.
- The database supports all system operations.

Step 9: Role of Fuel Delivery Agent

- The delivery agent receives the order.
- Collects fuel and delivers it to the user location.

Step 10: Role of Admin

- The admin monitors all activities.

5.2.3 CLASS DIAGRAM

A Class Diagram is a type of structural diagram in UML that represents the structure of a system by showing its classes, attributes, methods, and relationships between classes. It helps in understanding how different parts of the system are organized and how they interact with each other. In this project, the class diagram illustrates the main components such as user, admin, fuel order, payment, and delivery, along with their relationships.

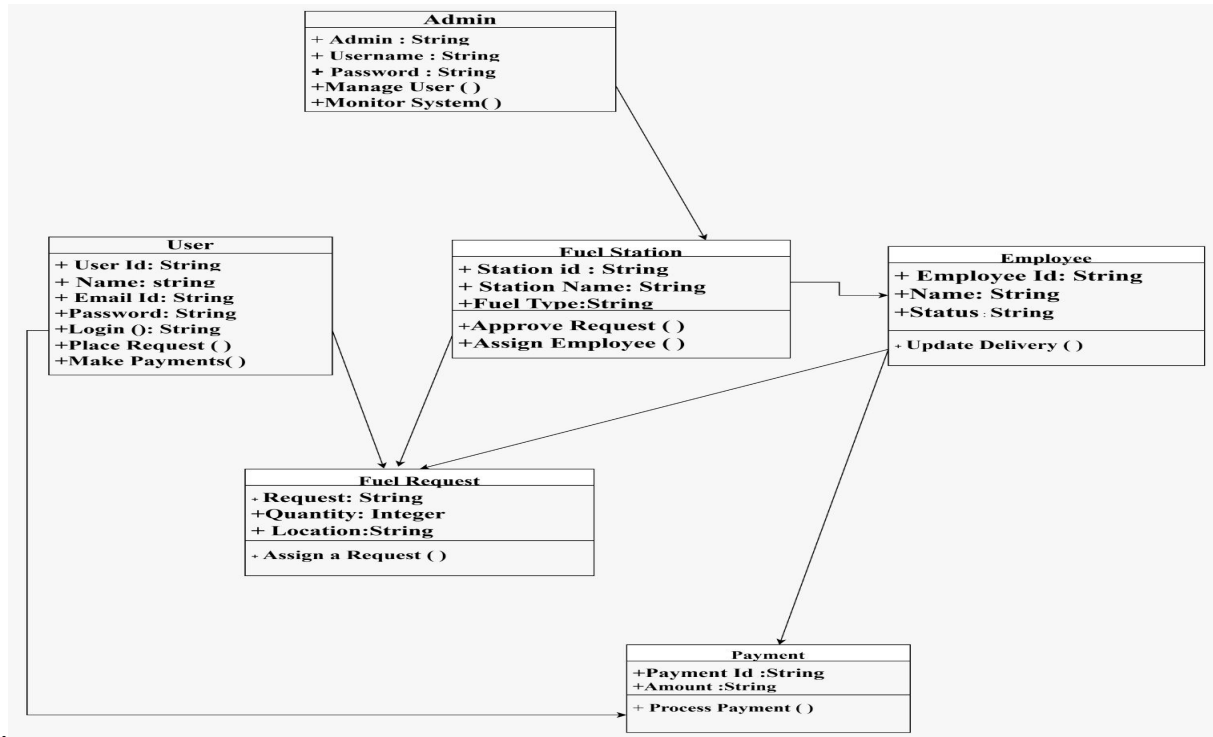


Fig 5.2.3 Class Diagram

5.2.4 ACTIVITY DIAGRAM

An Activity Diagram is a graphical representation that shows the flow of activities or actions in a system. It describes how different processes are carried out step by step, including decision points, parallel activities, and the overall workflow. Activity diagrams are useful for understanding the dynamic behaviour of a system and how tasks are executed from start to end. In this project, the activity diagram represents the complete flow of the fuel delivery process, from user login to final delivery. The activity diagram begins with the user accessing the system and performing registration or login. Once authenticated, the user searches for available fuel types and proceeds to place an order by selecting the required fuel and delivery location. The system then processes the request and checks for fuel availability. If the fuel is available, the system sends a notification to the fuel delivery agent; otherwise, the process may stop or prompt the user to try again. After the delivery agent accepts the request, the system confirms the order and directs the user to the payment process. The user completes the payment using online methods, and the system verifies the transaction. If the payment is successful, the delivery process begins, and the agent delivers the fuel to the specified location. Finally, the system updates the order status as completed. Throughout this workflow, the admin monitors and manages system operations to ensure smooth functioning.

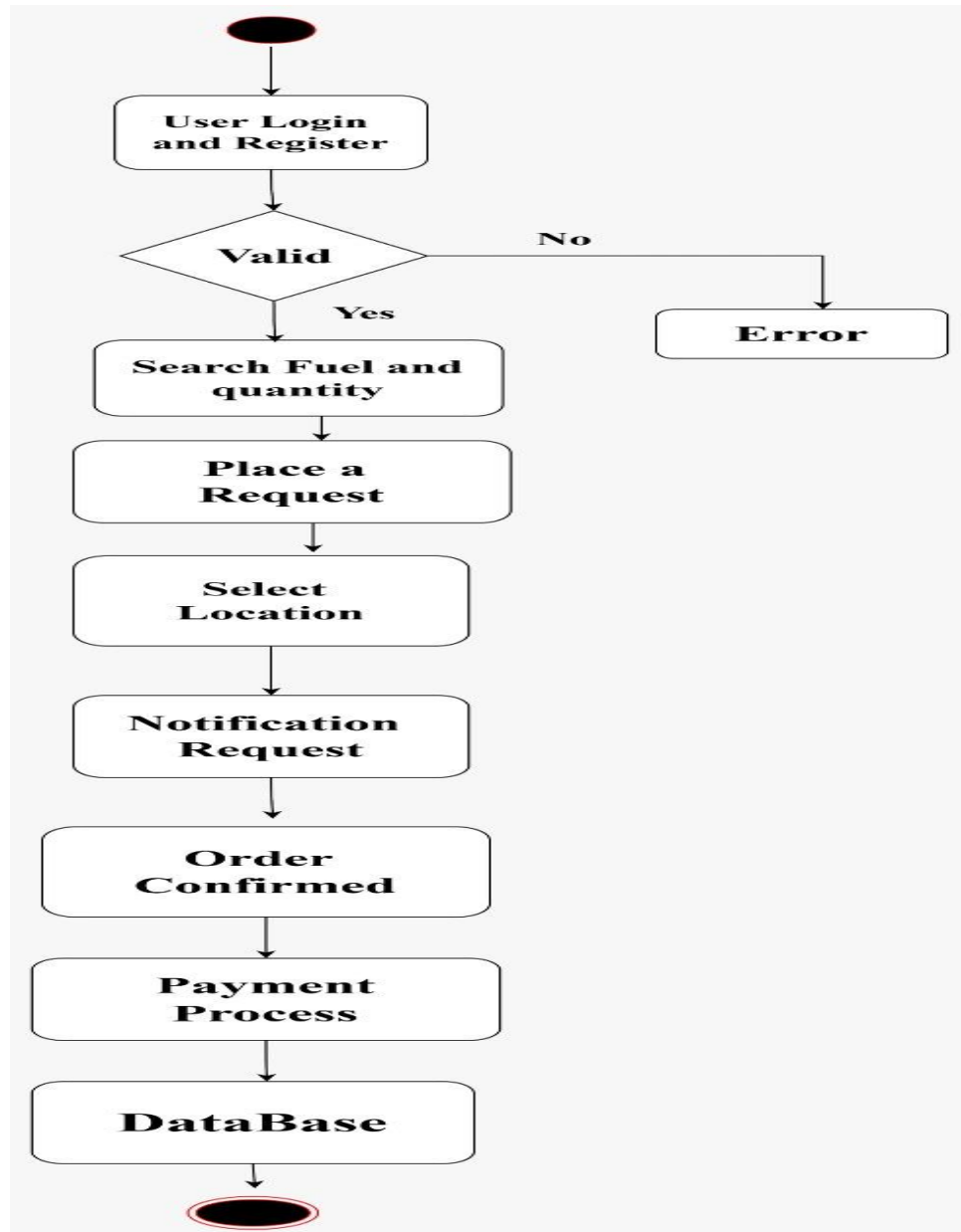


Fig 5.2.4 Activity Diagram

5.2.5 SEQUENCE DIAGRAM

A Sequence Diagram is a type of interaction diagram that shows how different components of a system interact with each other in a specific sequence over time. It represents the order of messages exchanged between actors such as the user, system, admin, and fuel delivery agent. In this project, the sequence diagram illustrates the step-by-step flow of operations from user login to fuel delivery, including order processing and payment.

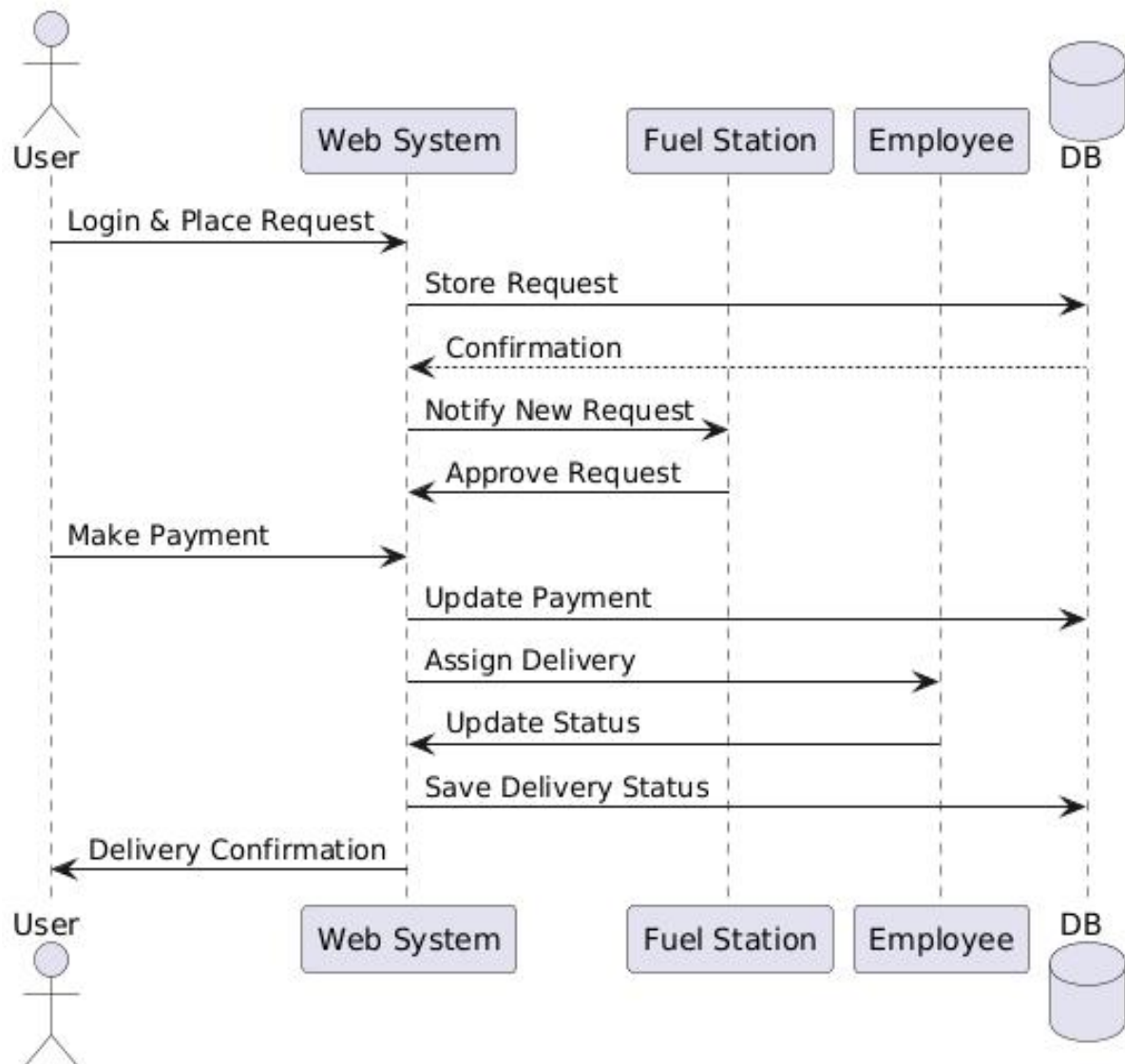


Fig 5.2.5 Sequence Diagram

CHAPTER - 06

IMPLEMENTATION

6.1 SAMPLE CODE

```
Admin.py
from flask import *
from database import *
admin=Blueprint('admin',__name__)

@admin.route('/adminhome',methods=['get','post'])
def adminhome():
    return render_template('adminhome.html')

@admin.route('/adminview_user',methods=['get','post'])
def adminview_user():
    data = {}
    q = "select * from Users"
    res = select(q)
    data['users'] = res
    return render_template('adminview_user.html',data=data)

@admin.route('/adminview_fuelstations',methods=['get','post'])
def adminview_fuelstations():
    data = {}
    q = "select * from fuelstations inner join login ON fuelstations.login_id = login.login_id
where status!='Accept';"
    res = select(q)
    data['fuel'] = res
    data['count'] = len(res)
    if 'id' in request.args:
id=request.args['id']
        q="update login set status='Accept' where login_id='%s' "%(id)
        update(q)
```

```
return redirect(url_for('admin.adminview_fuelstations'))
elif 'id1' in request.args:
    id1=request.args['id1']
    q="update login set status='Reject' where login_id='%s' "%(id1)
    update(q)
return redirect(url_for('admin.adminview_fuelstations'))
return render_template('adminview_fuelstations.html',data=data)

@admin.route('/adview_appr_fuelstations',methods=['get','post'])
def adview_appr_fuelstations():
    data = {}
    q = "select * from fuelstations inner join login ON fuelstations.login_id = login.login_id where status='Accept';"
    res = select(q)
    data['Afuel'] = res
    return render_template('adview_appr_fuelstations.html',data=data)

@admin.route('/adminadd_fuelcateg',methods=['get','post'])
def adminadd_fuelcateg():
    if 'submit' in request.form:
        categoryname=request.form['cname']
        q="select * from fuel_categorys where categoryname='%s' "%(categoryname)
        res=select(q)
        if len(res)>0:
            flash("The Fuel Category Is Already Exists")
        else:
            q="insert into fuel_categorys values(null,'%s' "%(categoryname)
            insert(q)
            flash("Category Added")
            return redirect(url_for('admin.adminview_fuelcateg'))

    return render_template('adminadd_fuelcategory.html')
```

```
@admin.route('/adminview_fuelcateg',methods=['get','post'])
def adminview_fuelcateg():
    data = {}
    if 'action' in request.args:
        action = request.args['action']
        id = request.args['id']
    else:
        action = None
    if action == 'delete':
        q = "delete from fuel_categorys where category_id='%s'" % (id)
        delete(q)
        return redirect(url_for('admin.adminview_fuelcateg'))
    if action == 'update':
        q = "select * from fuel_categorys where category_id='%s'" % (id)
        res = select(q)
        data['updatecat'] = res
    if 'update' in request.form:
        categoryname = request.form['cname']
        q = "update fuel_categorys set categoryname='%s' where category_id='%s'" %
(categoryname, id)
        update(q)
        return redirect(url_for('admin.adminview_fuelcateg'))
    q = "select * from fuel_categorys"
    res = select(q)
    data['Cat'] = res
    return render_template('adminview_fuelcategorys.html',data=data)

@admin.route('/adminview_newfb',methods=['get','post'])
def adminview_newfb():
    data = {}
    q = "select * from feedback inner join users using(user_id)"
    res = select(q)
    data['fb'] = res
```

```

j = 0
for i in range(1, len(res) + 1):
    if 'submit' + str(i) in request.form:
        reply = request.form['reply' + str(i)]
        q = "UPDATE feedback SET reply='%s' WHERE feedback_id='%s'" %
(reply, res[j]['feedback_id'])
        update(q)
        flash("send message")
        return redirect(url_for('admin.adminview_newfb'))
    j = j + 1
return render_template('adminview_newfeedback.html',data=data)
@admin.route('/adminview_repliedfb',methods=['get','post'])
def adminview_repliedfb():
    data = {}
    q = "select * from feedback inner join users using(user_id) "
    res = select(q)
    data['fb'] = res
    return render_template('adminview_repliedfeedback.html',data=data)
fuelstaion.py
from flask import *
from database import *
fuel=Blueprint('fuel',__name__)
@fuel.route('/fuelhome',methods=['get','post'])
def fuelhome():
    return render_template('fuelhome.html')
@fuel.route('/addfuel',methods=['get','post'])
def addfuel():
    data = {}
    ids = session['login_id']
    q = "select * from fuelstations where login_id='%s'" % (ids)
    res = select(q)
    data['fs']= res

```

```

if 'submit' in request.form:
    categ = request.form['Category']
    district = request.form['district']
    city = request.form['city']
    amount = request.form['amount']
    mobile = request.form['mobile']
    available=request.form['available']

    q = "insert into fuel values(null,(select fuelstation_id from fuelstations where
login_id='%s'),'%s','%s','%s','%s','%s','%s') " % (ids,categ, district, city, amount, mobile, available )
    insert(q)
    flash("Fuel Added")

    q = "select * from fuel_categorys"
    res1 = select(q)
    data['cat'] = res1
    return render_template('fuel_addfuel.html',data=data)
    @fuel.route('/viewfuel',methods=['get','post'])
def viewfuel():
    data={}

ids = session['login_id']
    q = "select * from fuel inner join fuelstations using(fuelstation_id) inner join
fuel_categorys using(category_id) where fuelstation_id=(select fuelstation_id from fuelstations
where login_id='%s') ""%(ids)
    res1 = select(q)
    data['fuel'] = res1
    if 'action' in request.args:
        action = request.args['action']
        id = request.args['id']
    else:
        action = None

    if action == 'delete':
        q = "delete from fuel where fuel_id='%s'" % (id)
        delete(q)
        return redirect(url_for('fuel.viewfuel'))

    if action=='update':
        q="select * from fuel where fuel_id='%s'""%(id)
        res=select(q)
        data['updatefuel']=res

```

```

        if 'update' in request.form:
            avail=request.form['available']
            amount=request.form['amount']
            q="update fuel set available='%s',amount='%s' where
fuel_id='%s'"%(avail,amount,id)
            update(q)
            return redirect(url_for('fuel.viewfuel'))
        return render_template('fuel_viewfuel.html',data=data)

@fuel.route('/fuelrequests',methods=['get','post'])
def fuelrequests():
    data = {}
    ids = session['login_id']
    q = "select * from request inner join fuel using(fuel_id) inner join fuelstations
using(fuelstation_id) where req_status='Pending' and fuelstation_id=(select fuelstation_id from
fuelstations where login_id='%s')"%(ids)
    res = select(q)
    data['req'] = res
    data['count']=len(res)
    if 'id' in request.args:
        id=request.args['id']
        q="update request set req_status='Accepted' where request_id='%s'"%(id)
        update(q)
        return redirect(url_for('fuel.fuelrequests'))
    return render_template('fuel_fuelrequests.html',data=data)

@fuel.route('/ap_fuelrequests',methods=['get','post'])
def ap_fuelrequests():
    data = {}
    ids = session['login_id']
    q = "select * from request inner join fuel using(fuel_id) inner join fuelstations
using(fuelstation_id) where req_status!='Pending' and fuelstation_id=(select fuelstation_id from
fuelstations where login_id='%s')"%(ids)
    res = select(q)
    data['req'] = res
    return render_template('fuel_apr_fuelrequests.html',data=data)

@fuel.route('/fuel_profile',methods=['get','post'])
def fuel_profile():
    data = {}
    ids = session['login_id']
    q = "SELECT * FROM `fuelstations` WHERE login_id='%s'" % (ids)
    res = select(q)
    data['my'] = res
    if 'action' in request.args:
        action = request.args['action']
        id = request.args['id']

```

```

else:
    action = None
if action=='update':
    q="select * from fuelstations where login_id='%s'"%(ids)
    res=select(q)
    data['updatecat']=res

if 'update' in request.form:
    mobile=request.form['mobile']
    q="update fuelstations set mobile='%s' where login_id='%s'"%(mobile,ids)
    update(q)
    return redirect(url_for('fuel.fuel_profile'))
return render_template('fuel_profile.html',data=data)

@fuel.route('/fuel_addemployees',methods=['get','post'])
def fuel_addemployees():
    ids = session['login_id']
    if 'submit' in request.form:
        name=request.form['name']
        add=request.form['address']
        email=request.form['email']
        mob=request.form['mobile']
        psw=request.form['password']
        q = "select * from login where username='%s'" % (email)
        res = select(q)
        if len(res) > 0:
            flash("Your Email ID Already Exists ")
        else:
            q = "insert into login values(null,'%s','%s','employee',null)" % (email, psw)
            result = insert(q)
            q = "insert into employees values(null,'%s',(select fuelstation_id from
fuelstations where login_id='%s'),'%s','%s','%s'))" % (result,ids, name,add,mob)
            insert(q)
            flash("Employee Registration Successfully Completed")
        return render_template('fuel_addemployees.html')
@fuel.route('/fuel_viewemployees',methods=['get','post'])
def fuel_viewemployees():
    data={}
    ids = session['login_id']
    q = "select * from employees inner join login using(login_id) where fuelstation_id= (select
fuelstation_id from fuelstations where login_id='%s') "%(ids)
    res1 = select(q)
    data['emp'] = res1
    if 'action' in request.args:
        action = request.args['action']
        id = request.args['id']
    else:
        action = None

```



```

if action == 'delete':
    q = "delete from login where login_id='%s'" % (id)
    q1 = "delete from employees where login_id='%s'" % (id)
    delete(q)
    delete(q1)
    return redirect(url_for('fuel.fuel_viewemployees'))
return render_template('fuel_viewemployees.html',data=data)

@fuel.route('/fuel_chpassword',methods=['get','post'])
def fuel_chpassword():
    return render_template('fuel_chpassword.html')

@fuel.route('/fuel_assignworks',methods=['get','post'])
def fuel_assignworks():
    data = {}
    ids = session['login_id']
    q="select * from request inner join fuel using(fuel_id) inner join fuelstations
using(fuelstation_id) where fuelstation_id=(select fuelstation_id from fuelstations where
login_id='%s')and request.pay_status='Paid' "% (ids)
    res = select(q)
    data['req'] = res

    q="select * from employees where fuelstation_id=(select fuelstation_id from fuelstations
where login_id='%s')"%(ids)
    res1=select(q)
    data['emp']=res1

    j = 0
    for i in range(1, len(res) + 1):
        if 'submit' + str(i) in request.form:
            employee = request.form['employee']
            q = "UPDATE request SET employee_id='%s' WHERE
request_id='%s'" % (employee, res[j]['request_id'])
            update(q)
            q = "insert into works values(null,'%s','%s','Pending')" %
(res[j]['request_id'], employee)
            insert(q)
            flash("Work Assigned for the Employee")
            return redirect(url_for('fuel.fuel_assignworks'))
        j = j + 1

    return render_template('fuel_assignworks.html',data=data)
index.html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">

```

```
<title>Online Fuel Delivery System</title>
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<!-- Bootstrap CSS -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">

<!-- Font Awesome -->
<link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.2/css/all.min.css"
rel="stylesheet">

<style>
  body {
    font-family: 'Poppins', sans-serif;
  }

  /* Hero Section */
  .hero-section {
    background: url('https://images.unsplash.com/photo-1605152276897-4f618f831968') no-
repeat center center/cover;
    height: 100vh;
    color: white;
    display: flex;
    align-items: center;
    text-align: center;
    position: relative;
  }

  .hero-section::before {
    content: "";
    position: absolute;
    width: 100%;
    height: 100%;
    background: rgba(0,0,0,0.6);
  }

  .hero-content {
    position: relative;
    z-index: 2;
  }

  .navbar {
    background: #000 !important;
  }

  .feature-icon {
    font-size: 40px;
    color: #ff5722;
```

```

    }

    .footer {
        background: #111;
        color: white;
        padding: 20px 0;
    }

    .btn-primary {
        background-color: #ff5722;
        border: none;
    }

    .btn-primary:hover {
        background-color: #e64a19;
    }
</style>
</head>

<body>

<!-- ===== HEADER ===== -->
<nav class="navbar navbar-expand-lg navbar-dark fixed-top">
    <div class="container">
        <a class="navbar-brand fw-bold" href="#">🛢 FuelExpress</a>

        <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav">
            <span class="navbar-toggler-icon"></span>
        </button>

        <div class="collapse navbar-collapse justify-content-end" id="navbarNav">
            <ul class="navbar-nav">
                <li class="nav-item"><a class="nav-link active" href="/">Home</a></li>
                <li class="nav-item"><a class="nav-link" href="user_register">Register</a></li>
                <li class="nav-item"><a class="nav-link" href="fuel_register">Fuel Station</a></li>
                <li class="nav-item"><a class="nav-link" href="login">Login</a></li>
            </ul>
        </div>
    </div>
</nav>

<!-- ===== HERO SECTION ===== -->
<section class="hero-section">
    <div class="container hero-content">
        <h1 class="display-4 fw-bold">Online Fuel Delivery System</h1>
        <p class="lead">Order fuel anytime, anywhere. Fast, Safe & Reliable doorstep delivery.</p>
    </div>
</section>

```

```
<a href="user_register" class="btn btn-primary btn-lg mt-3">Order Now</a>
</div>
</section>
```

```
<!-- ===== FEATURES ===== -->
```

```
<section class="py-5">
  <div class="container text-center">
    <h2 class="mb-4 fw-bold">Why Choose Us?</h2>
    <div class="row">

      <div class="col-md-4">
        <i class="fa-solid fa-truck feature-icon"></i>
        <h4 class="mt-3">Fast Delivery</h4>
        <p>Fuel delivered to your location within minutes.</p>
      </div>

      <div class="col-md-4">
        <i class="fa-solid fa-shield-halved feature-icon"></i>
        <h4 class="mt-3">Safe & Secure</h4>
        <p>Certified fuel stations and secure online payments.</p>
      </div>

      <div class="col-md-4">
        <i class="fa-solid fa-clock feature-icon"></i>
        <h4 class="mt-3">24/7 Service</h4>
        <p>Available anytime for emergency fuel needs.</p>
      </div>

    </div>
  </div>
</section>
```

```
<!-- ===== HOW IT WORKS ===== -->
```

```
<section class="bg-light py-5">
  <div class="container text-center">
    <h2 class="mb-4 fw-bold">How It Works</h2>
    <div class="row">

      <div class="col-md-4">
        <h5><img alt="user icon" data-bbox="238 841 258 858"/> Register/Login</h5>
        <p>Create your account and login securely.</p>
      </div>

    </div>
  </div>
</section>
```

```
<div class="col-md-4">
  <h5>2 Place Order</h5>
  <p>Select fuel type and enter delivery location.</p>
</div>

<div class="col-md-4">
  <h5>3 Doorstep Delivery</h5>
  <p>Fuel delivered safely to your vehicle.</p>
</div>

</div>
</div>
</section>
<!-- ===== FOOTER ===== -->
<footer class="footer text-center">
  <div class="container">
    <p class="mb-1"> Online Fuel Delivery System </p>
    <small>Developed using HTML5, CSS3, Bootstrap & JavaScript</small>
  </div>
</footer>
<!-- Bootstrap JS -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>

</body>
</html>
```

CHAPTER – 07

TESTING

7.1 SYSTEM TESTING

Testing and validation are essential phases in software development to ensure that the system works correctly according to requirements. The testing process verifies whether each module of the Online Fuel Delivery and Management System performs accurately and whether the complete system functions without errors. Validation ensures that the system meets user needs, provides expected outputs, and maintains reliability during real-time usage.

7.1 Testing Process

The testing process involves checking individual modules as well as the integrated system to identify errors, improve performance, and ensure smooth functionality. Different testing methods are applied to verify system behaviour under various conditions.

1. Unit Testing

Unit testing focuses on testing individual modules or functions independently. Each module such as user login, fuel request handling, payment processing, and delivery update is tested separately to verify correctness.

2. Integration Testing

Integration testing verifies the interaction between modules after combining them. It ensures that data flows correctly between components such as user module, fuel station module, and payment module.

3. System Testing

System testing evaluates the complete application as a whole to ensure that all modules operate together correctly. The system is tested under real conditions to check performance, reliability, and workflow accuracy.

4. Functional Testing

Functional testing confirms that the system fulfils user requirements and expected behaviour. It ensures that the output generated matches the intended objectives of the project.

5. Security Testing

Security testing ensures that unauthorized users cannot access protected resources.

6. Performance Testing

Performance testing checks system responsiveness and stability when multiple users access

7.2 TYPES OF TESTING

1. Unit Testing

Unit testing is the process of testing individual modules or components of the system in isolation to ensure that each unit performs correctly. It focuses on verifying the smallest parts of the application such as functions, methods, or classes. Each module like user authentication, energy data collection, device control, and payment processing is tested separately. For example, the login module is tested to check whether it correctly validates user credentials, and the device control module is tested to ensure that devices turn ON/OFF as expected.

2. Integration Testing

Integration testing is performed to verify the interaction between different modules and ensure that they work together as a complete system. It focuses on data flow and communication between components. The frontend, backend, and database are integrated and tested to ensure smooth communication and proper data transfer. It also verifies that inputs are correctly processed and stored, and that system components interact without errors.

3. Performance Testing

Performance testing evaluates how well the system performs under different workloads and conditions. It checks system speed, responsiveness, scalability, and stability. The system is tested to ensure it can handle real-time data, multiple user requests, and continuous operations without delays or failures. It also ensures that the system performs efficiently under heavy load conditions.

4. Acceptance Testing

Acceptance testing is conducted to verify whether the system meets user requirements and is ready for deployment. It is performed from the user's perspective by testing all functionalities such as login, monitoring, control operations, and payment processes. This testing ensures that the system is user-friendly, reliable, and meets the expected requirements.

5. Black Box Testing

Black box testing is a method where the internal structure of the system is not considered. It focuses only on testing functionality based on inputs and outputs. Inputs such as user credentials, requests, and commands are provided, and outputs are verified to ensure correct system behaviour. This testing ensures that the system functions correctly

according to user expectations.

6. White Box Testing

White box testing involves testing the internal logic, structure, and code of the system. It requires knowledge of programming and focuses on verifying code correctness. It ensures proper execution of logic, loops, and conditions, and checks all possible code paths. This testing helps in improving code quality, identifying hidden errors, and ensuring efficient system performance.

7.2 TESTCASES

Test Case ID	TC01
Module	User Module
Objective	Verify user registration
Input	Valid user details
Expected Result	User account created successfully
Actual Result	Account created and stored in database
Status	Pass

Test Case 1 – User Registration

Test Case ID	TC02
Module	Authentication
Objective	Verify login functionality
Input	Correct username and password
Expected Result	User redirected to dashboard
Actual Result	Login successful
Status	Pass

Test Case 2 – User Login

Test Case ID	TC03
Module	Fuel Request
Objective	Verify request creation
Input	Fuel type, quantity, location
Expected Result	Request stored and visible to fuel station
Actual Result	Request saved successfully
Status	Pass

Test Case 3 – Fuel Request Submission

Test Case ID	TC04
Module	Place order
Objective	Approve fuel request
Input	Place order without entering location
Expected Result	Request status updated to approved
Actual Result	Place order without entering location
Status	Fail

Test Case 4 – Fuel Station Approval

Test Case ID	TC05
Module	Payment Module
Objective	Validate payment processing
Input	Payment with insufficient balance
Expected Result	Payment success and status update
Actual Result	Payment filled but no error message displayed
Status	Fail

Test Case 5 – Payment Processing

CHAPTER – 08

RESULTS

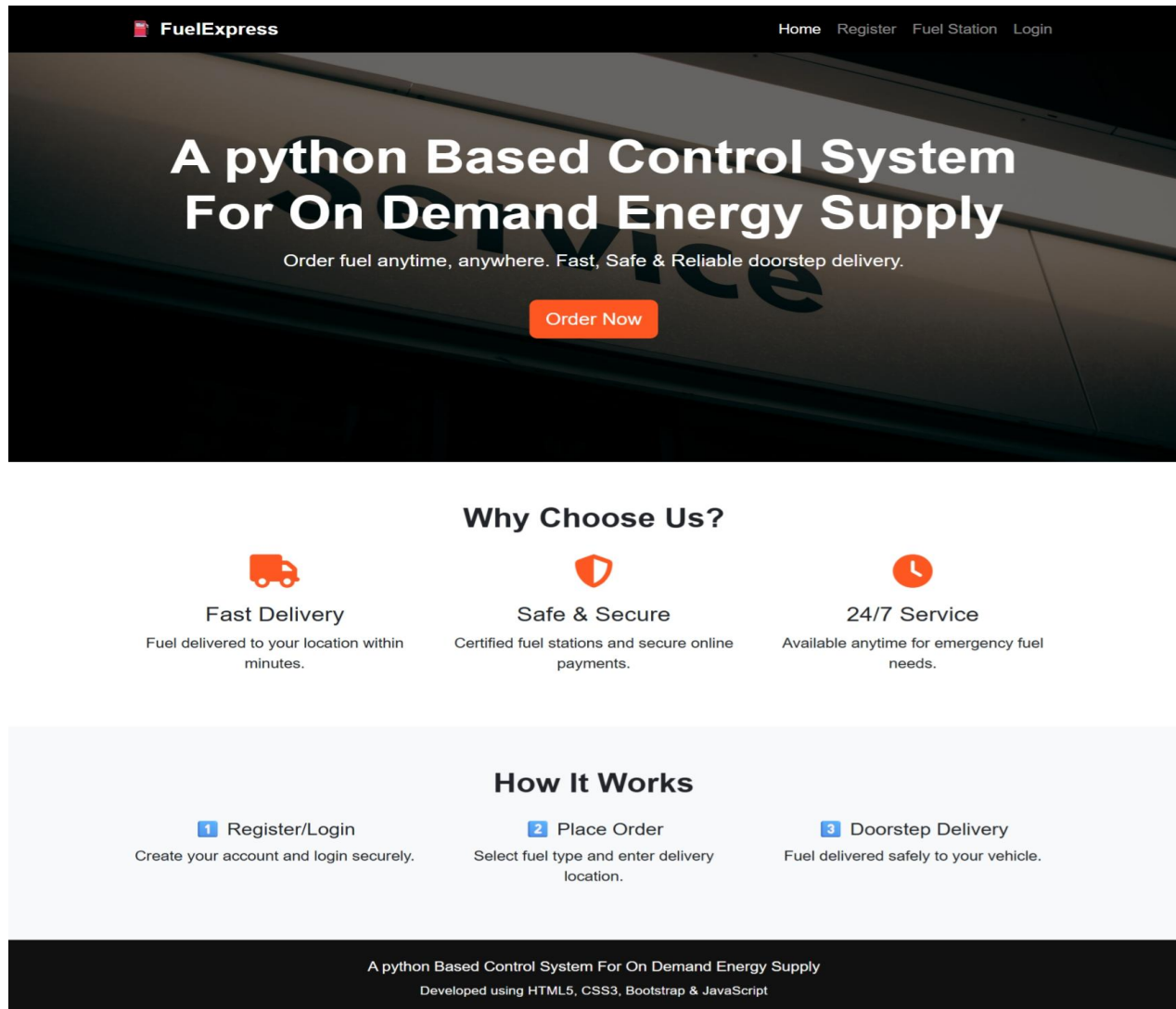


Fig 8.1.1 Home Page

This is a homepage of a Python-based fuel delivery system that allows users to order fuel online with fast, safe, and reliable doorstep service.

It highlights key features like quick delivery, secure payments, 24/7 availability, and a simple three-step ordering process.

FuelExpress Home Register Login

User Login
Access your Fuel Delivery Account

Email Address
indianoll@gmail.com

Password

Login Now

Don't have an account? [Register Here](#)

A python Based Control System For On Demand Energy Supply
Fast • Safe • Reliable Doorstep Fuel Delivery
A python Based Control System For On Demand Energy Supply

Fig 8.1.2 User Login

This screen shows a pagination interface that helps users navigate through multiple pages of content easily.

FuelExpress Home Register Login

User Registration

Full Name
Enter Full Name

Date of Birth
dd-mm-yyyy

District
Select Your District

City
Enter City

Street
Enter Street

Pincode
6 Digit Pincode

Mobile Number
10 Digit Mobile Number

Email
indianoll@gmail.com

Password

Register Now

A python Based Control System For On Demand Energy Supply
Fast • Safe • Reliable Doorstep Fuel Delivery
A python Based Control System For On Demand Energy Supply

Fig 8.1.3 Register Page

This is a user registration page for the Fuel Express system where new users can create an account to access fuel delivery services.

A Python based control system for on demand energy supply

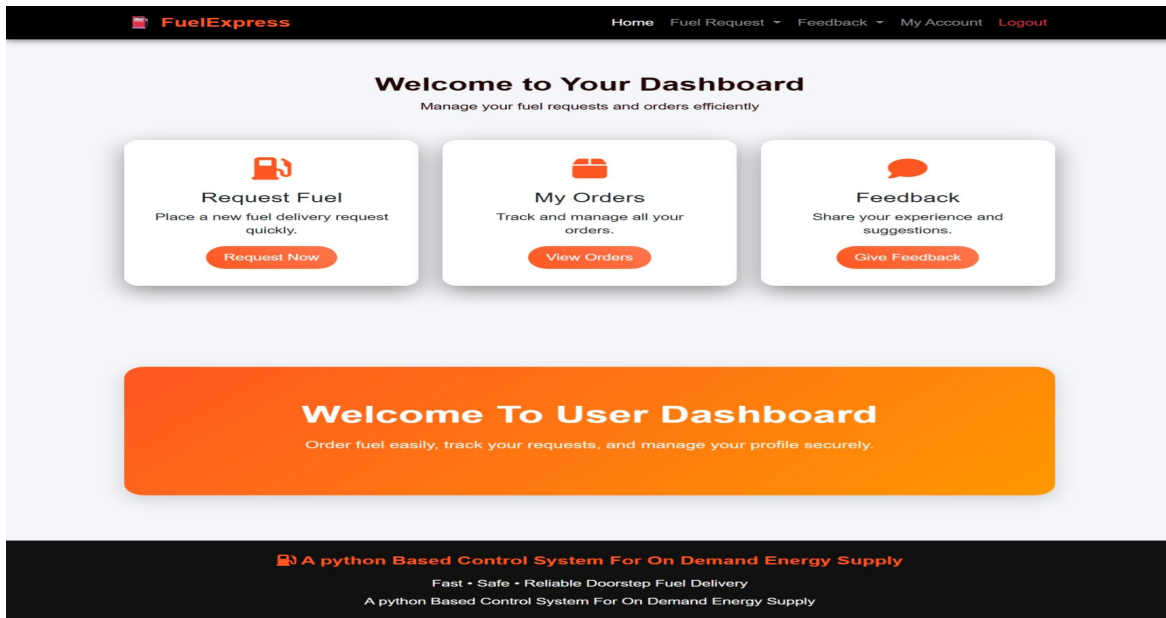


Fig 8.1.4 User Dashboard

A user dashboard interface for a fuel delivery system, allowing users to request fuel, track orders, and provide feedback efficiently.

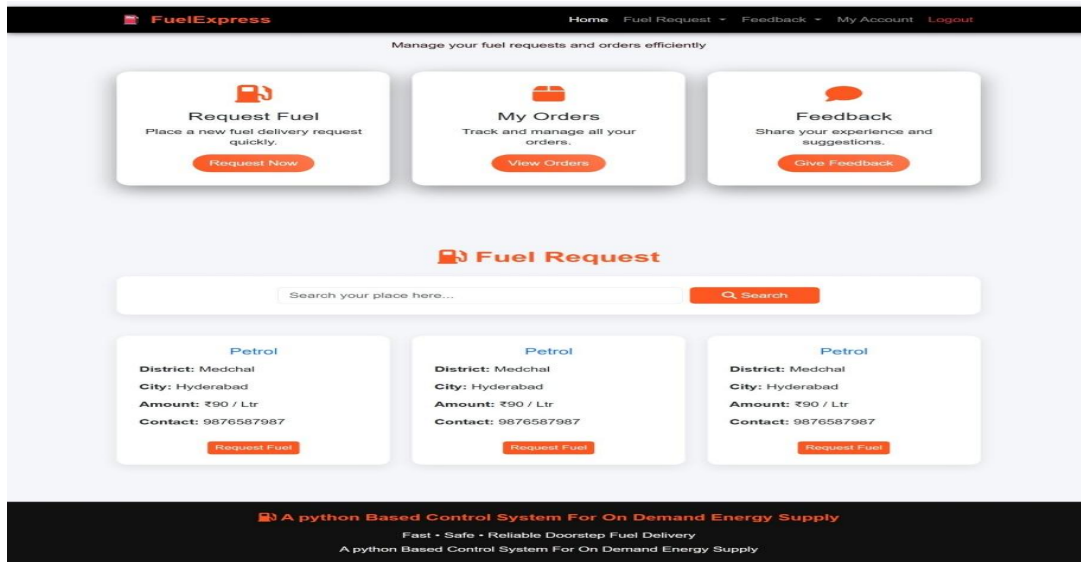


Fig 8.1.5 Fuel Request

A fuel request interface that lets users search locations, view fuel details, and place delivery requests

A Python based control system for on demand energy supply

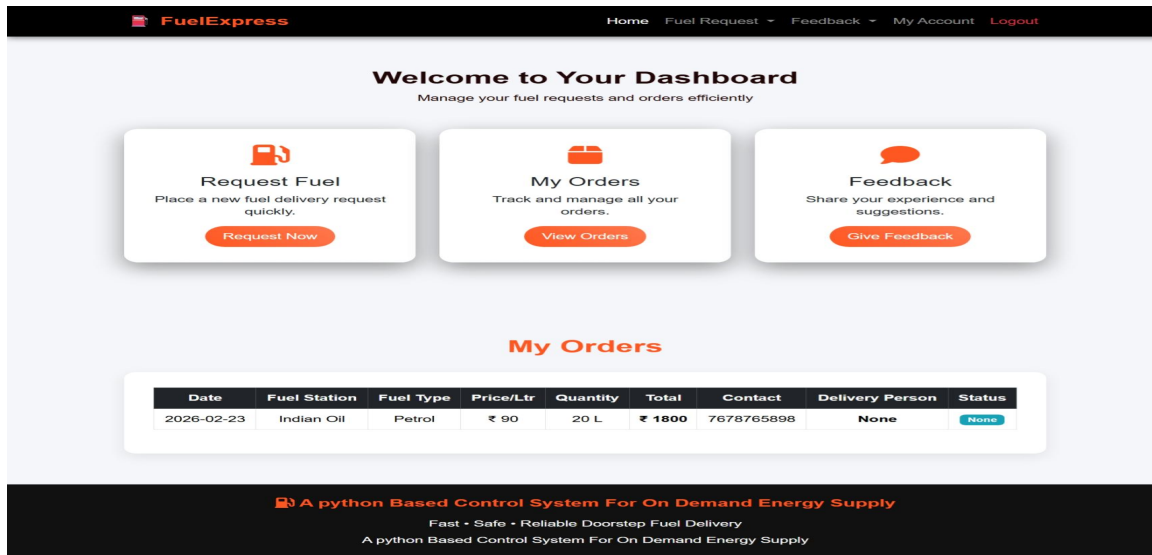


Fig 8.1.6 My Orders

A “My Orders” dashboard page that displays users’ fuel order details, including date, type, quantity, price, and delivery status.

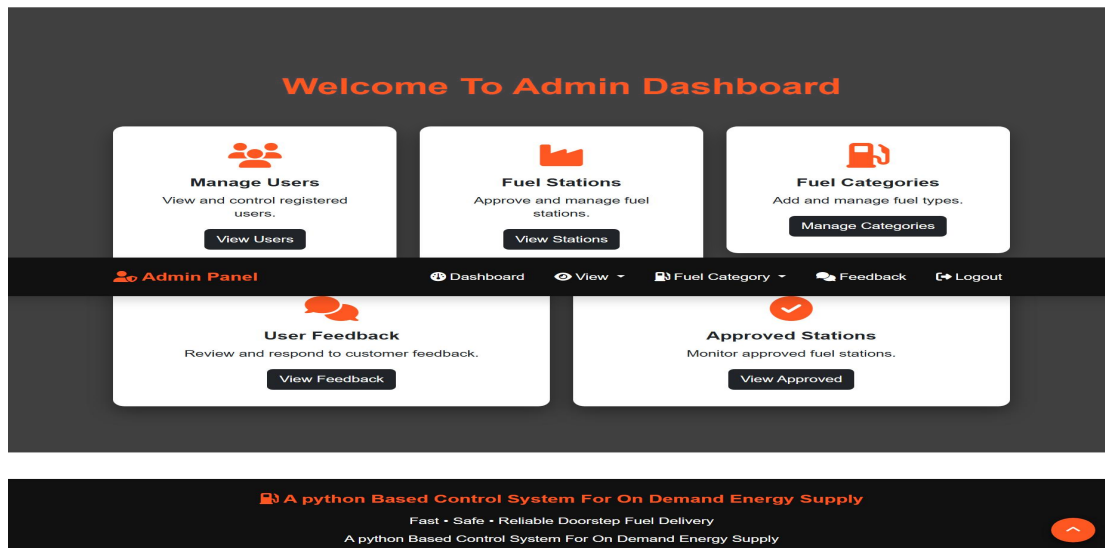


Fig 8.1.7 Admin Dashboard

An admin dashboard interface that enables management of users, fuel stations, categories, feedback, and approved stations.

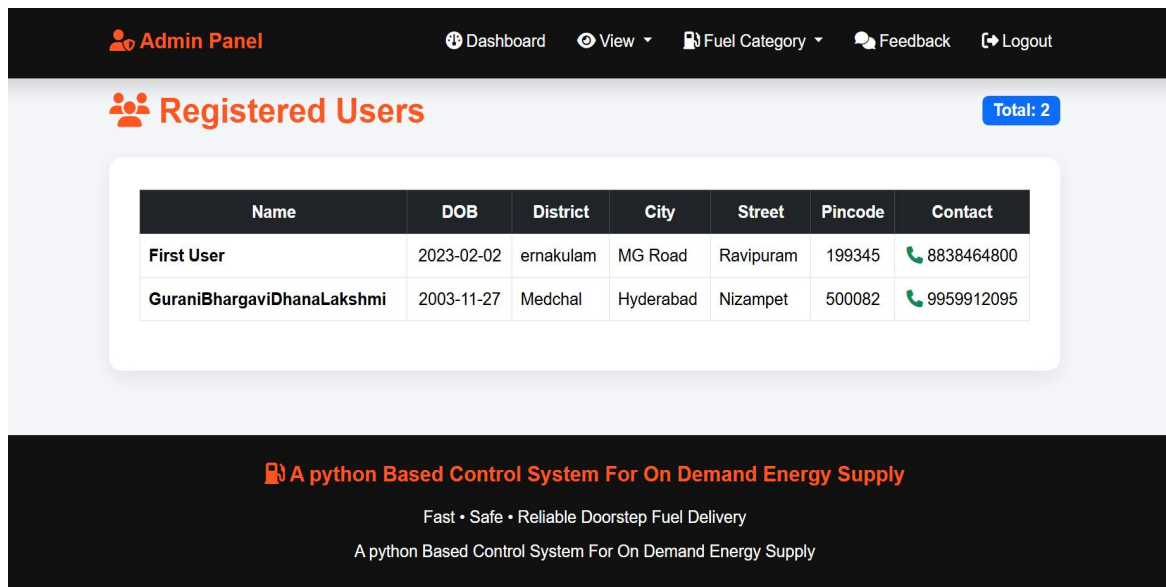


Fig 8.1.8 Registration

This figure shows the admin panel displaying a list of registered users in the Python-based on-demand energy supply system.

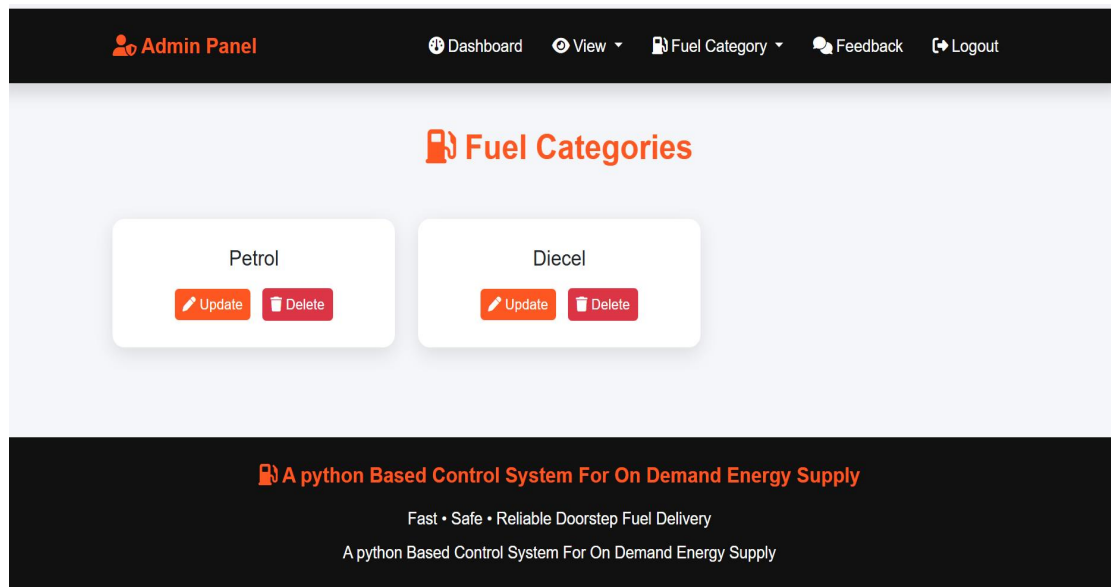


Fig 8.1.9 Admin Panel

This figure shows the admin panel interface used to manage fuel categories like petrol and diesel in the system.

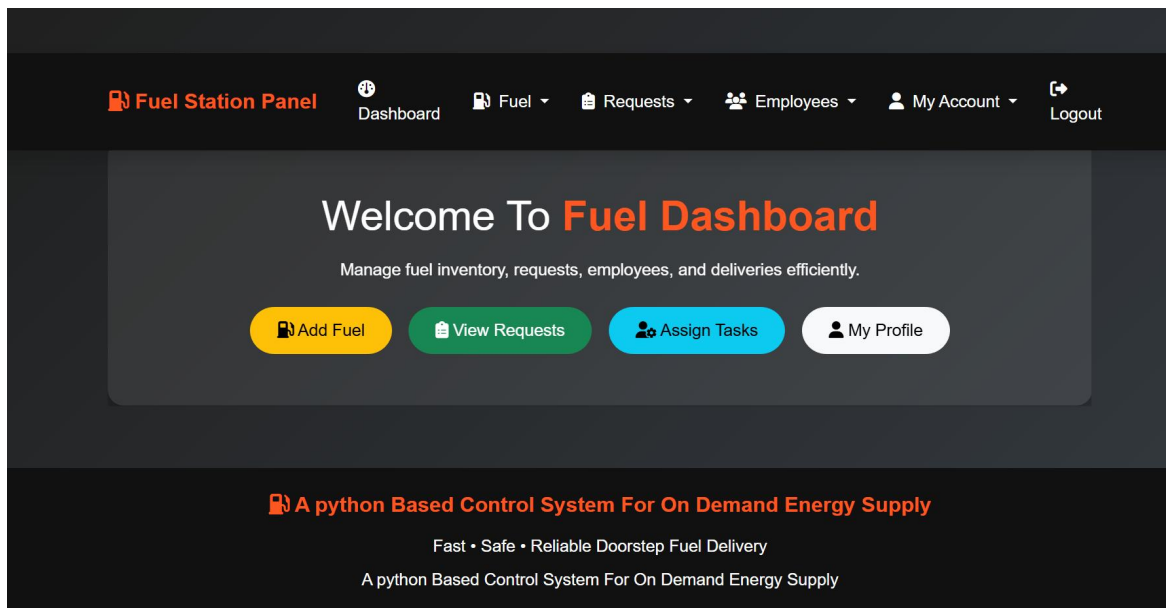


Fig 8.1.10 Fuel Dashboard

This figure shows the fuel station dashboard interface used to manage fuel inventory, requests, employees, and deliveries.

The image shows a web form titled 'Fuel Station Panel' with a sub-header 'Add Fuel'. The form contains several input fields: 'Fuel Station Name' (with 'Indian Oil' entered), 'Type of Fuel' (with 'Select Fuel Type' as a placeholder), 'District' (with 'Medchal' entered), 'City' (with 'Hyderabad' entered), 'Amount Per Liter (₹)' (with '90' entered), 'Contact Number' (with '9876587987' entered), and 'Availability' (with 'Available' as a placeholder). A red 'Add Fuel' button is located at the bottom of the form. Below the form, there is a footer section with the text 'A python Based Control System For On Demand Energy Supply' and 'Fast • Safe • Reliable Doorstep Fuel Delivery'.

Fig 8.1.11 Fuel Panel

This figure shows the fuel station panel interface used to add fuel details such as type, price, location, and availability.

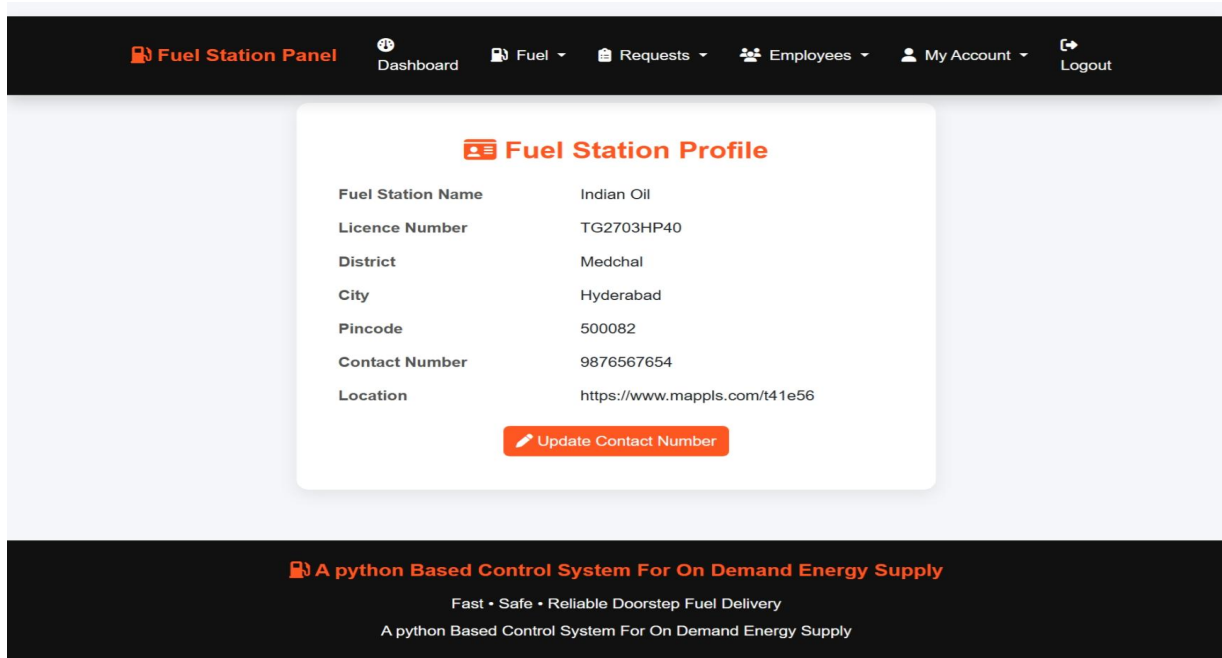


Fig 8.1.12 Fuel Station Profile

This figure shows the fuel station profile page displaying details like name, license number, location, and contact information.

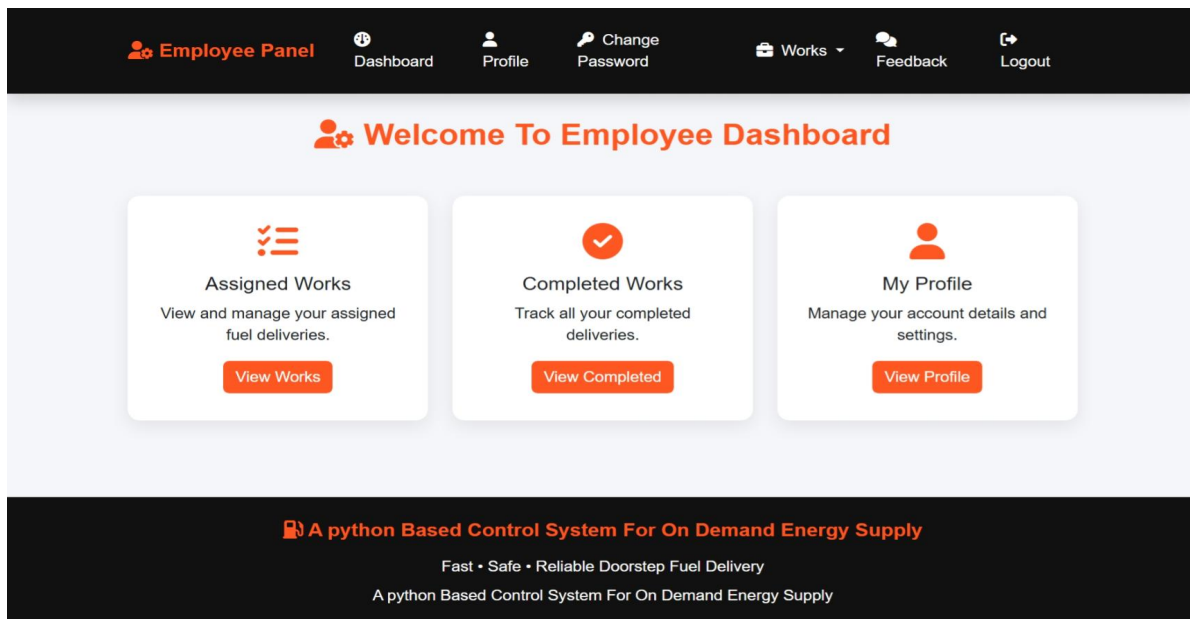


Fig 8.1.13 Employee Dashboard

This figure shows the Employee Dashboard of a Python-based On-Demand Energy Supply Control System, where employees can view assigned work, completed tasks.

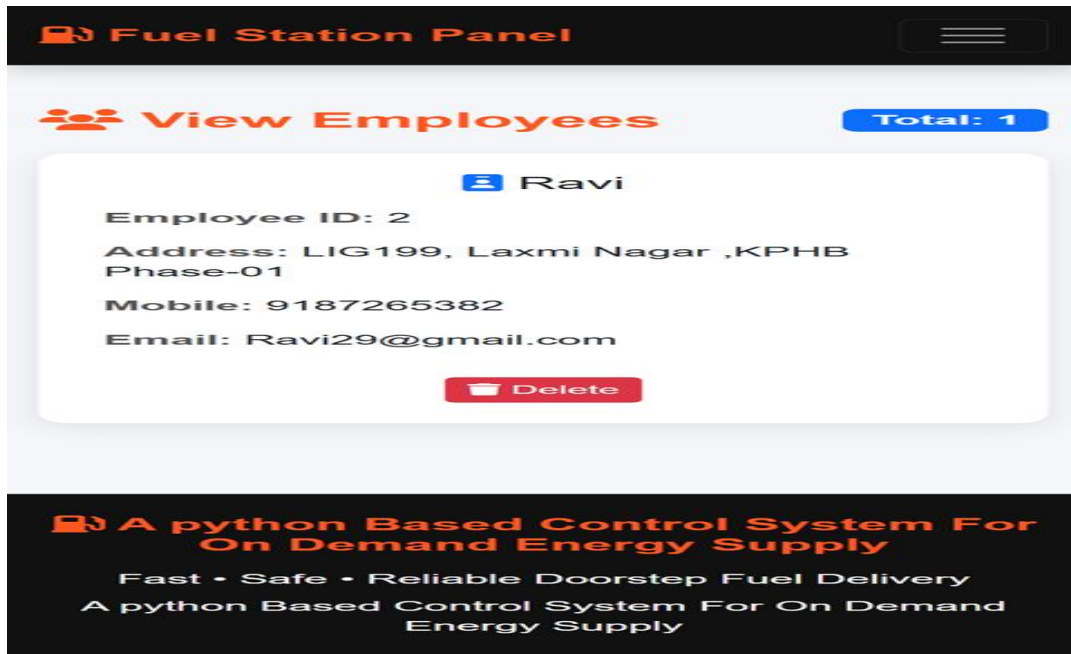


Fig 8.1.14 Employee

It's a digital dashboard for a fuel station management system that displays employee details and operational information in a structured interface.

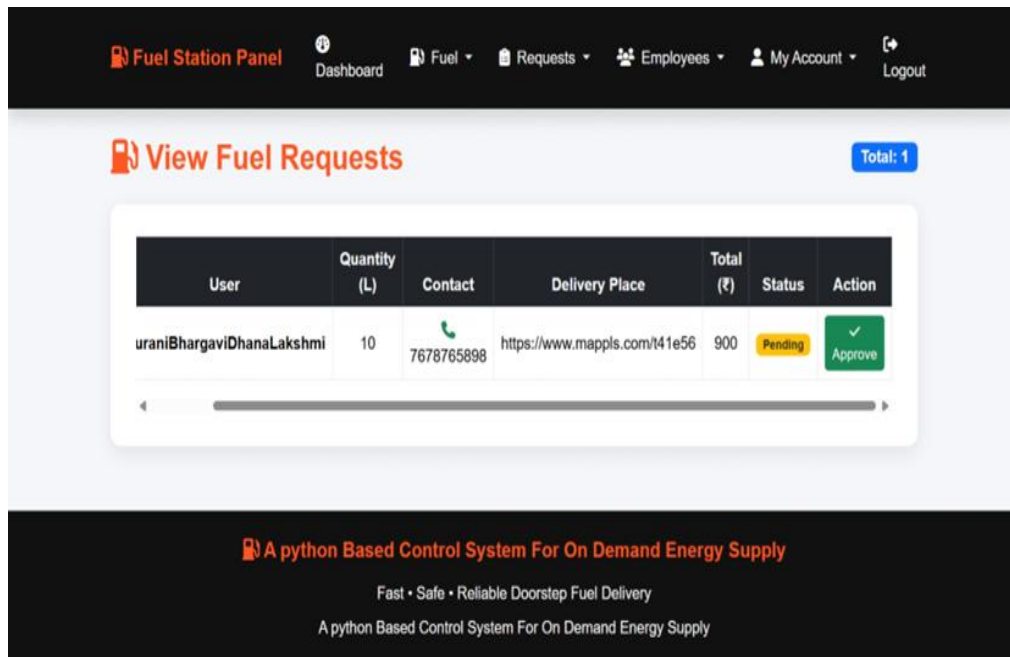


Fig 8.1.15 Fuel Request Approved

It's a web-based fuel station management dashboard that displays customer fuel requests with details like quantity, contact, delivery location, cost, and approval status.

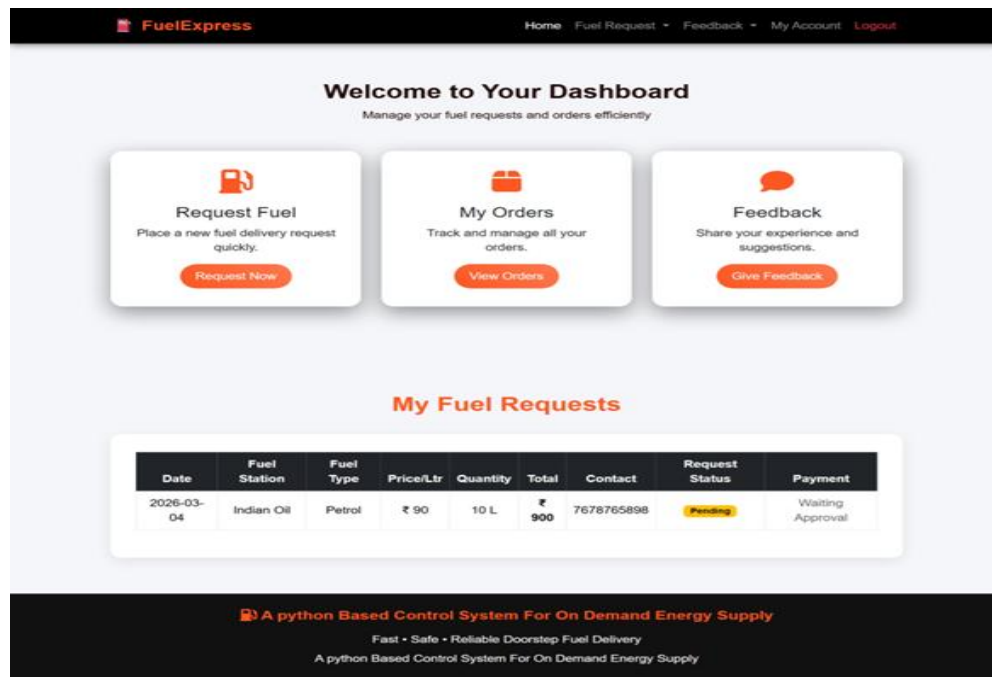


Fig 8.1.16 Fuel Request Pending

It's a user-friendly dashboard for a fuel delivery system that lets customers request fuel, track orders, and give feedback, while also showing pending fuel requests with details like station, type, price, quantity, and payment status.

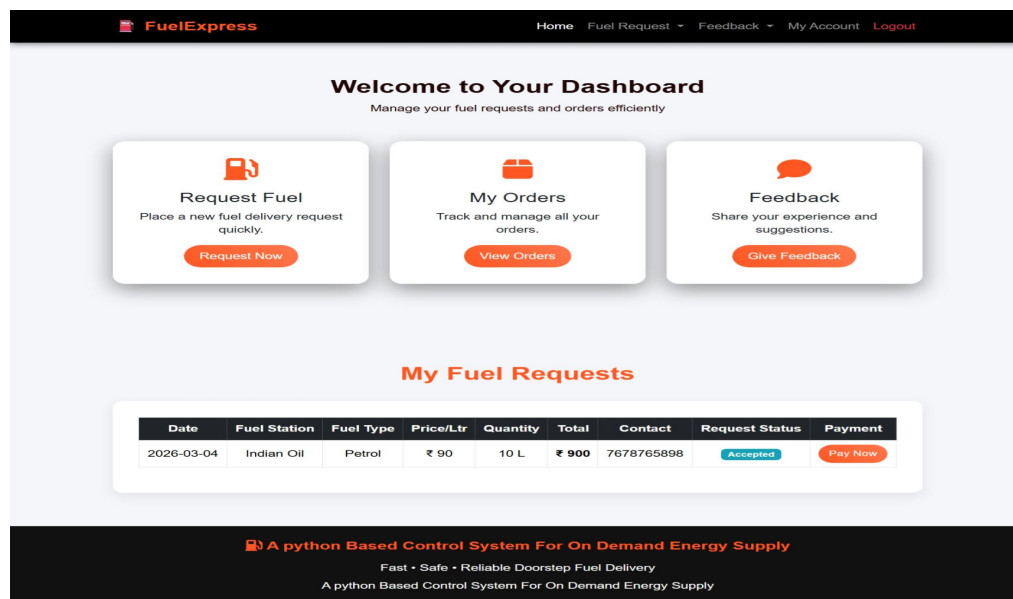


Fig 8.1.17 Order confirmed

It's a Fuel Express dashboard that lets users request fuel, track orders, give feedback, and manage accepted fuel deliveries with payment options.

A Python based control system for on demand energy supply

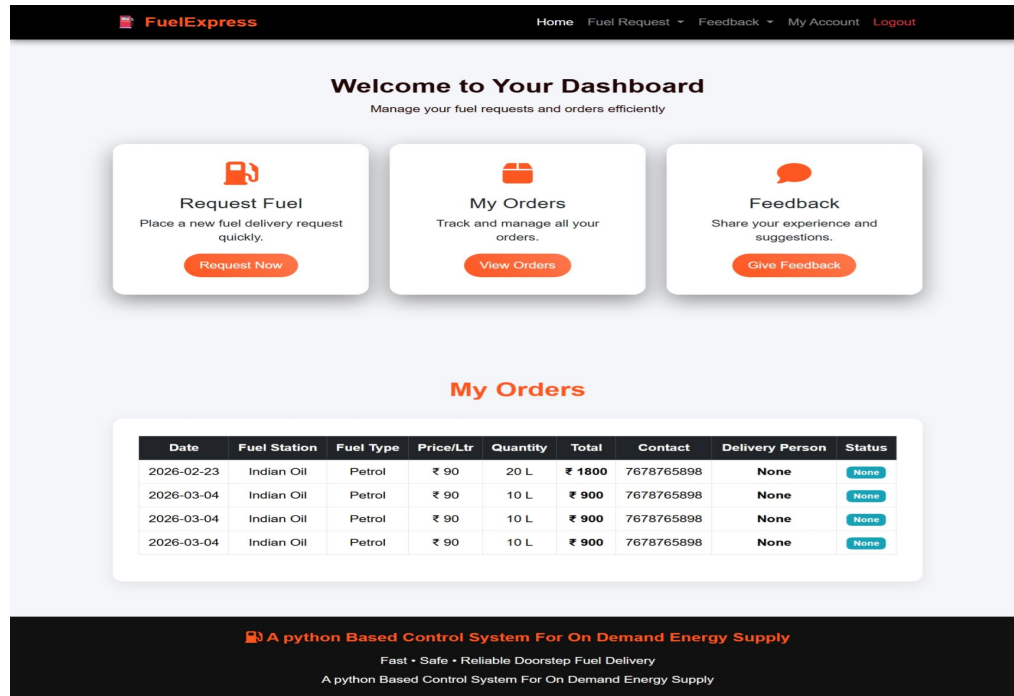


Fig 8.1.18 Order Page

It's a Fuel Express dashboard that lets users request fuel, track multiple orders, and view order details like station, type, cost, and status in one place.

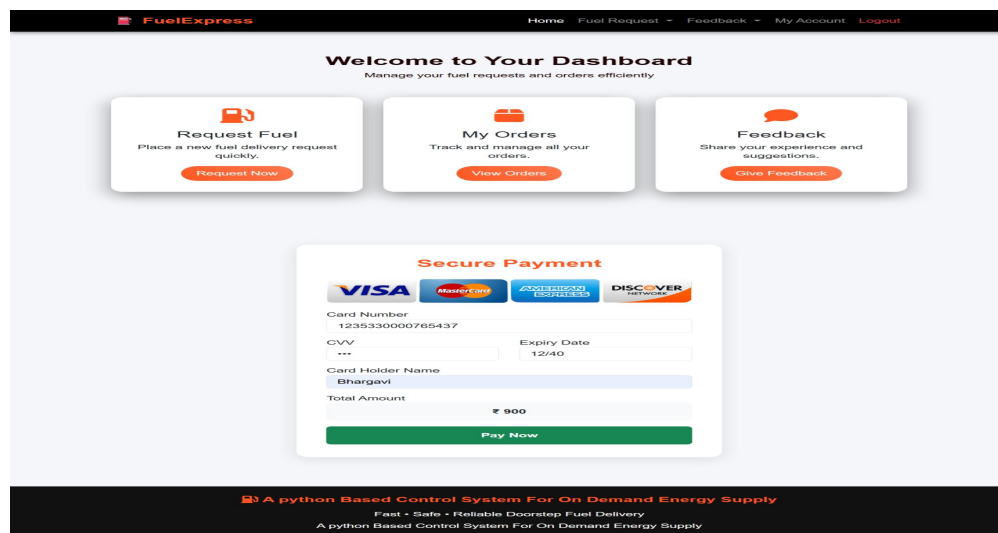


Fig 8.1.19 Payment Process

It's a Fuel Express dashboard that combines fuel request management, order tracking, feedback options, and a secure online payment system.

CHAPTER – 09

CONCLUSION

A Python Based Control System for On Demand Energy Supply successfully provide a modern digital solution to overcome the limitations of traditional fuel distribution methods. The project introduces a structured web-based platform that connects users, fuel stations, delivery employees, and administrators within a single integrated system. By enabling online fuel ordering and doorstep delivery, the system significantly improves convenience, operational efficiency, and service reliability. The implementation of role-based modules ensures smooth coordination among stakeholders. Users can easily request fuel and track delivery status, fuel stations can manage requests and inventory efficiently, employees can update delivery progress digitally, and administrators can monitor the entire workflow from a centralized dashboard. The integration of secure authentication, automated request management, and digital payment handling reduces manual errors and enhances transparency. Using technologies such as HTML, CSS, JavaScript, Python Flask, and MySQL, the system demonstrates how modern web technologies can be applied to solve real-world logistical challenges. The modular architecture ensures maintainability and scalability, allowing future improvements without major redesign. Overall, the project achieves its objective of creating an efficient, reliable, and user-friendly fuel delivery management system while improving customer satisfaction and operational control. A Python Based Control System for On Demand Energy Supply successfully provides a modern digital solution to overcome the limitations of traditional fuel distribution methods. The project introduces a structured web-based platform that connects users, fuel stations, delivery employees, and administrators within a single integrated system. By enabling online fuel ordering and doorstep delivery, the system significantly improves convenience, operational efficiency, and service reliability. Users can easily request fuel and track delivery status, fuel stations can manage requests and inventory efficiently, employees can update delivery progress digitally, and administrators can monitor the entire workflow from a centralized dashboard. The integration of secure authentication, automated request management, and digital payment handling reduces manual errors and enhances transparency and real-world challenges.

CHAPTER – 10

FUTURE ENHANCEMENT

In the future, Python Based Control System for On Demand Energy Supply can be further enhanced by integrating advanced technologies to improve efficiency, safety, and user experience. A mobile application can allow users to place fuel orders anytime and anywhere. Real-time GPS tracking can help customers monitor delivery vehicles and receive accurate delivery updates. Artificial Intelligence and Machine Learning can analyze customer usage patterns, predict fuel demand, and optimize delivery routes. The system can also support multiple digital payment options such as UPI, wallets, and net banking for secure transactions. And also monitor fuel levels, detect leakage, and prevent fuel theft. Additional features like emergency fuel delivery, subscription plans, and multi-language support can further enhance accessibility and customer satisfaction. Further improvements can include the development of advanced dashboards with real-time analytics and graphical representations for better understanding of energy consumption patterns. Security features can be strengthened by implementing encryption techniques and multi-factor authentication to protect user data. The system can also be integrated with cloud platforms to improve scalability, storage, and remote accessibility. Moreover, features like voice control and smart automation can be added to enhance user convenience. Intelligent alert systems can notify users about unusual energy usage or system faults. With these enhancements, the system can become more efficient, secure, scalable, and user-friendly, meeting future technological demands. Future enhancements refer to the possible improvements that can be made to the system to increase its efficiency, functionality, and user experience. The system can be further developed by integrating advanced IoT devices for more accurate and real-time data collection. Artificial Intelligence techniques can be implemented to predict energy demand and optimize energy distribution more effectively. A mobile application can also be developed to provide users with easy access and control over the system from anywhere. Additionally, the system can be enhanced by integrating renewable energy sources such as solar and wind power, promoting sustainable energy usage. In addition, the system can be upgraded to support predictive maintenance, which helps in identifying faults in devices before failure occurs. Integration with smart home systems and other automation platforms can also be implemented for better connectivity. Furthermore, the system can be extended to support large-scale environments such as industries and smart cities, making it more scalable and adaptable to future technological advancements.

CHAPTER – 11

REFERENCES

- [1]. Asai Kiran et al. (2024), Online Fuel Delivery System Using IoT (IRJMETs), Designed an IOT enabled automated petrol and microcontrollers ,2024.
- [2]. Srushti Gunthe, et al. (2023), “Fuel Delivery Application”, Uses Android-based online fuel delivery platform using the K-Nearest Neighbor (KNN) algorithm for location-based service optimization,2023
- [3]. Gowri and R. Sathish Kumar, (2022) “Smart Automatic Petrol Pump System Based on Internet of Things”, This work focuses on an IoT-based smart petrol pump system enabling real-time delivery tracking and optimized routing.
- [4]. R. Khatun, et al. (2021) “Vehicle Fuel Activities Monitoring System Using IoT,” The paper presents an IoT-driven fuel monitoring and vehicle tracking system for efficient fuel usage analysis.
- [5]. Pallets Project (2023). *Flask Official Documentation – Python Web Framework Concepts*.
- [6].Oracle Cooperation (2023). *MySQL Documentation – Relational Database Management System*.
- [7]. World Wide Web Consortium (2022). HTML5, CSS3, and JavaScript Web Development Standards.
- [8]. Object management Group (2021). UML Modelling and System Design Concepts.
- [9]. Computer Science Research Community (2020-2024). Research papers on online delivery management systems and web-based service platforms.